



Luxor university
Faculty of Computer and Information
Computer Science Department



**The Management Intelligent System
For Managing Individuals' Residence in Cities**

**Graduation Project
2024/2025**

**Under Supervision
Dr. Bassem Abd-El-Atty**

Submitted by

- | | |
|---------------------------|---------------------------|
| - Mohamed Badawy Sayed | - Mostafa Ahmed Hasan |
| - Mahmoud Ahmed Khalil | - Ahmed Nageh Abbas |
| - Nada Maaman Abdel Karim | - Basant Benyamen Barsoum |

Abstract

The Management Intelligent System For Managing Individuals' Residence in Cities, known as Semsary, is a web-based system designed with .NET, React, and Python (for machine learning integrations) to address the complex demands of short-term rental management in urban settings. Tailored for tenants, landlords, and families, Semsary streamlines the apartment rental experience by consolidating reliable listings, automating price assessments, and providing a secure booking environment.

Semsary tackles key challenges such as providing precise rental information, ensuring fair pricing, and connecting users to essential amenities. Through machine learning analysis, the platform recommends optimal rental prices and offers feedback to landlords on market standards, promoting transparency and helping renters make informed choices. This data-driven approach minimizes risk of fraud and ensures fair valuation, enabling users to find accommodations that align with their preferences and budget constraints.

Beyond a listing repository, Semsary's intuitive interface allows users to filter apartments by features like proximity to transportation and schools. Verified reviews foster informed decision-making, and location-based insights offer an enriched user experience. Secure, streamlined booking and payment options further simplify the rental process.

Semsary also features a dynamic administrative layer where managers handle advertising approvals, resolve disputes, and review tenant reports. Dedicated ID verification for both local and international users enhances trust and security on the platform. In cases of dissatisfaction, tenants can request refunds or rate properties, supporting accountability and enhancing service standards.

In conclusion, Semsary is positioned to drive positive change in urban rental management. By centralizing resources, integrating machine learning for equitable pricing, and enhancing user security and convenience, Semsary fosters a sustainable rental environment that aligns with the goals of sustainable urban growth and community well-being.

Table of contents

Abstract	2
Table of contents	3
Chapter 1 : System Overview	1
1.1 Motivation	2
1.2 Problem Statement	2
1.3 Overview	3
1.4 Objectives and scope	3
1.5 Target Audience and Expected Impact	4
Chapter 2 : Related Works	5
2.1 Airbnb (https://www.airbnb.com)	6
2.2 Propertyfinder (https://www.propertyfinder.eg)	7
2.3 Aqarmap (https://aqarmap.com.eg)	8
2.4 Bayut (https://www.bayut.com)	9
2.5 Aqaryamasr	10
Chapter 3 : Domain Analysis and Technique	11
3.1 Domain Analysis	12
3.1.1 Clients and users	12
3.1.2 The Environment	12
3.1.3 Tasks and actions currently being performed	13
3.2 Risks	14
3.3 Constrains	15
3.4 Project plan	16
3.5 Feasibility Study	17
3.5.1 Technical feasibility	17
3.5.2 Operational feasibility	18
3.5.3 Economic feasibility	19
3.6 Quality Assurance Plan	20
3.7 System Requirements	22
3.7.1 Functional Requirements	22
3.7.2 Non-Functional Requirements	23
3.8 Techniques and tools	24

3.8.1 Frontend Framework.....	24
3.8.2 Backend Development	26
3.8.3 Machine Learning Model	27
3.8.4 Third Party Services	28
Chapter 4 : Proposed System & Methodology	34
4.1 System Use-Cases	35
4.1.1 Landlord	35
4.1.2 Tenant	36
4.1.3 Admin Use-Case	37
4.1.4 Customer Service	37
4.2 Use Case Description (Use case scenario)	38
4.3 System Architecture	61
4.4 Analysis Class	61
4.4.1 State Diagram	61
4.4.2 Data flow diagram (Level Zero diagram)	61
4.4.3 Context Diagram	71
4.5 Activity diagrams	72
4.6 interaction class diagram	75
4.6.1 System diagram	75
4.6.2 Sequence Diagram	75
4.7 Design Class	75
4.7.1 Class Diagram	75
4.8 Database Schema	75
4.9 ER Diagram	75
4.10 Design Mockup	75
Chapter 5: Proposed System & Methodology	76
5.1 Conclusions	77
5.2 Future work	77
5.3 References	77

Chapter 1

System Overview

1.1 Motivation

In today's world, people frequently move to different cities for various reasons, such as pursuing university studies, new work opportunities, vacations, and more. In addition to the increase in the number of expatriates to all governorates of Egypt in the recent period. Despite their varied purposes, one essential need unites them: finding a suitable place to live.

For many of these individuals, especially expatriates and students, the rental process can be challenging. Options for short-term, flexible, and furnished rentals are often limited, and securing a safe, well-priced residence in an unfamiliar city is not always straightforward.

Our project addresses this need with a platform designed to support sustainable cities and communities. By enabling homeowners to list available rentals and allowing tenants to compare and book our solution simplifies the rental process for everyone involved. In doing so, we aim to facilitate a more accessible, trustworthy, and sustainable rental market that empowers individuals and contributes to local community development.

Our motivation stems from the urgent need to transform our countries into sustainable cities and communities. While there are many factors to achieve this goal, developing the process of renting apartments to make it easier and smarter is one of the first steps to achieve this goal.

1.2 Problem Statement

Ineffectiveness of available methods of displaying available apartments, making it difficult for tenants to find the best match for their needs. Without clear information on prices, apartment contents, and nearby facilities, tenants struggle to make informed decisions about where to live.

Exploitation by brokers of both the landlord and the tenant, they get a commission from both of them in exchange for bringing the tenant to the landlord. They also often use misleading information to convince the tenant of the apartment.

There is a lack of effective methods to ensure that the tenant's expectations are met, which exposes him to the risk of receiving an apartment that does not conform to the agreed upon specifications or even that this apartment does not actually exist on the ground. This discrepancy between what was promised and what was delivered can lead to dissatisfaction and trust issues in the rental process.

Many tenants face challenges in obtaining real, transparent feedback about apartments, leaving them vulnerable to misinformation or deceptive practices. Without reliable reviews or insights, tenants cannot confidently evaluate the quality of an apartment before committing to a rental.

Landlords often struggle to reach a large and relevant audience of potential tenants, limiting their ability to fill available apartments efficiently. Without effective tools to market their properties, landlords miss out on valuable opportunities to connect with suitable tenants who may be actively searching for housing options.

1.3 Overview

1.4 Objectives and scope

Our system, Semsary, is strategically designed to address key challenges in the short-term rental market, offering a streamlined, user-friendly, and data-driven system to facilitate apartment management and booking. By focusing on efficiency, security, and transparency, Semsary aims to revolutionize the urban rental experience for students, families, working individuals, and property owners alike.

Our primary objectives center on enhancing user experience, enabling data-backed rental pricing, simplifying booking processes, and ensuring informed decision-making. Through comprehensive apartment listings that detail essential features, amenities, and locations, Semsary empowers users to make well-informed choices, significantly reducing time spent on property searches. With advanced filtering options, users can effortlessly tailor their search to specific needs, including room count, rental type, and proximity to essential services such as transportation, schools, and markets.

Semsary also integrates machine learning capabilities to evaluate and recommend optimal rental prices based on input data, including room specifications, nearby services, and market trends. This approach provides landlords with pricing guidance aligned with market standards, while giving tenants confidence in fair and competitive pricing. The platform's data-driven pricing analysis supports transparency, minimizes the risk of fraud, and promotes financial fairness within the rental ecosystem.

In addition, the system enables seamless booking and secure payment options, which are essential for a smooth user experience. Property owners benefit from efficient request management, allowing them to review and respond to booking requests while ensuring exclusivity for each listing. A transparent request process also allows users to preview and cancel bookings within a specified time-frame, with funds held securely until completion or cancellation.

Semsary's comprehensive scope includes account creation, user interest customization, detailed transaction management, and a dynamic advertisement system, all contributing to an ecosystem that prioritizes sustainable urban living. By offering premium accounts, paid advertisements, and cancellation fees, Semsary not only enhances user experience but also ensures a sustainable revenue model.

1.5 Target Audience and Expected Impact

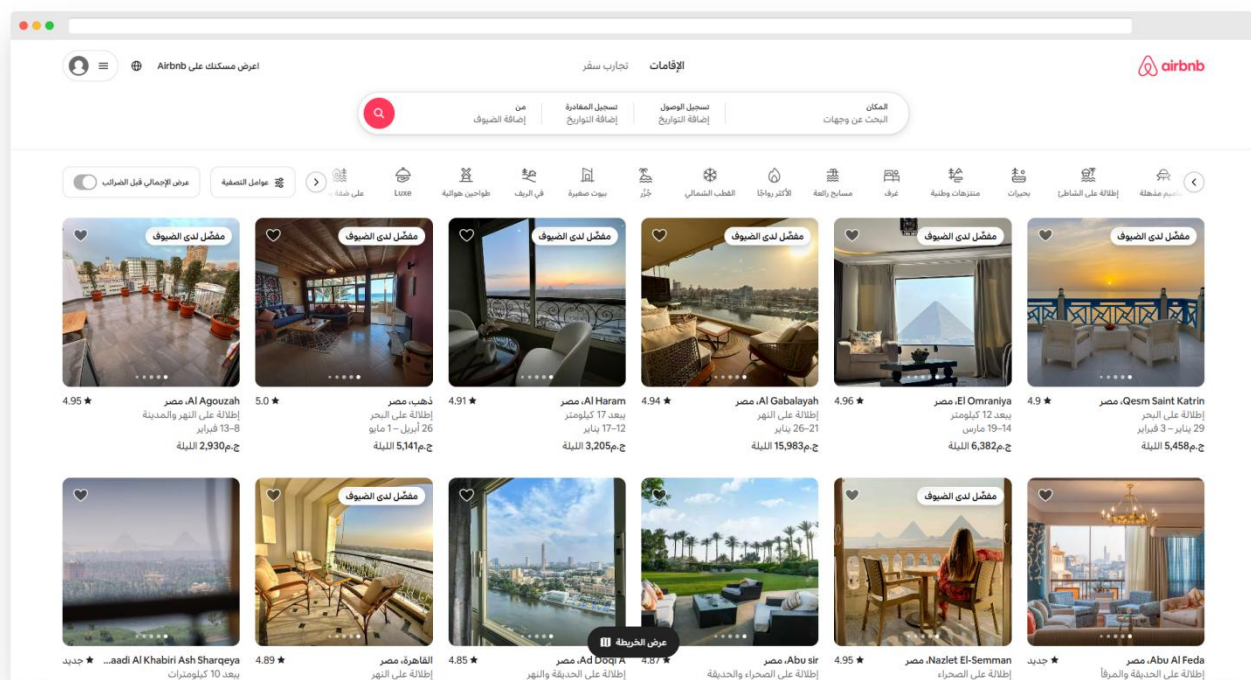
Our primary target audience includes Egyptian students, expatriate students, workers, Egyptian and expatriate families who are relocating to new cities for work, education, or personal reasons. These individuals often face challenges in finding short-term, flexible, and furnished rental accommodations that suit their needs. Additionally, landlords looking to rent out apartments or rooms to a wide range of tenants will also benefit from the platform. This includes homeowners in busy urban areas who require an easy and effective way to connect with potential tenants.

The expected impact of this platform is twofold: for tenants, it will make the process of finding reliable, affordable, and flexible housing options faster and more transparent, reducing the time and effort spent searching for suitable homes. For landlords, it will offer a more efficient way to connect with a wider pool of tenants, increasing the likelihood of renting out properties quickly. In the long term, the platform will contribute to building more sustainable communities by making housing access more equitable and transparent, promoting urban mobility, and supporting sustainable urban development.

Chapter 2

Related Works

2.1 Airbnb (<https://www.airbnb.com>)



Airbnb is a global online marketplace, founded in 2008 in San Francisco, that connects hosts offering short-term rentals with travelers seeking diverse accommodations, from shared spaces to unique stays like castles and yurts. Operating in over 220 countries, Airbnb revolutionized travel by providing flexible and personalized lodging options.

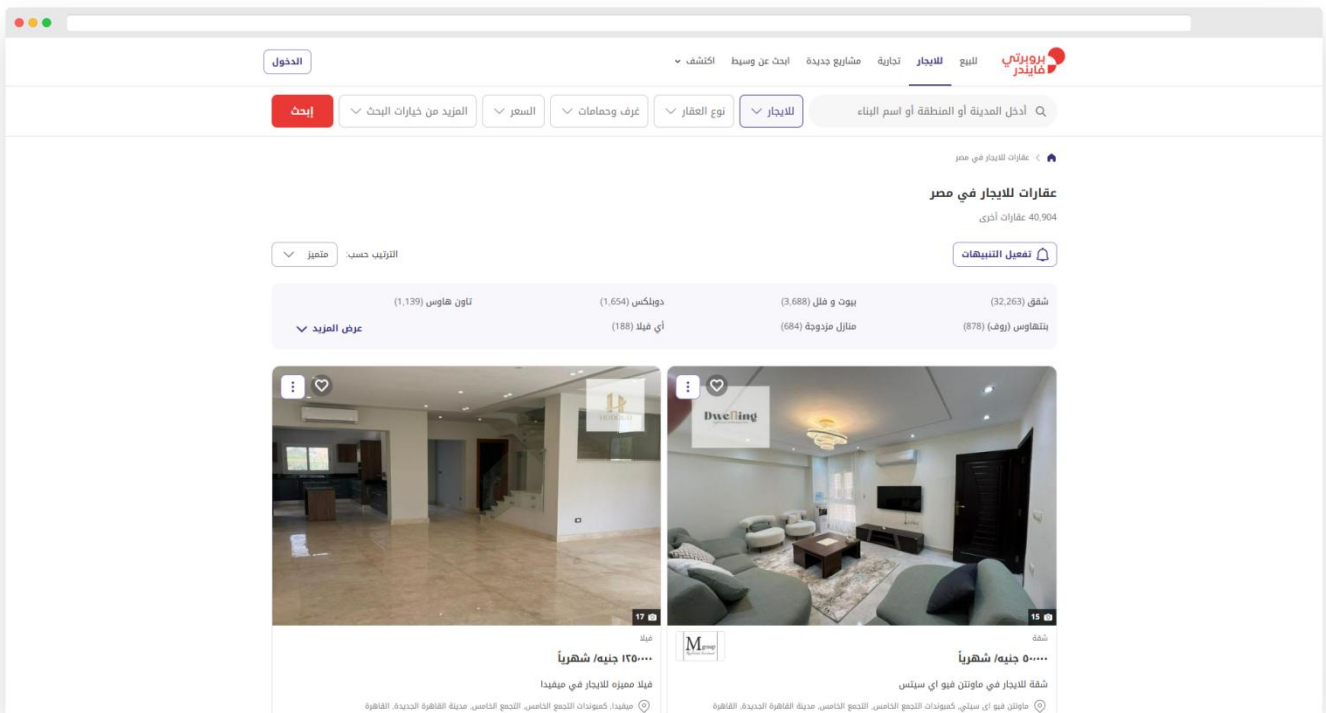
In addition to accommodations, Airbnb offers "Experiences," enabling travelers to participate in local activities hosted by residents, enhancing cultural immersion. The platform's peer-to-peer model charges service fees to both hosts and guests, creating a sustainable revenue stream.

Through innovative technology, transparency, and a focus on community and sustainability, Airbnb has transformed the hospitality industry, fostering global connections and redefining how people travel and explore the world.

Business Model

Airbnb uses a peer-to-peer model. It charges service fees to both hosts and guests for bookings made through the platform. Hosts benefit from a global audience for their properties, while guests enjoy a variety of accommodations often at more competitive prices than hotels.

2.2 Propertyfinder (<https://www.propertyfinder.eg>)



Property Finder was founded by Michael Lahyani in Dubai, initially launching as a printed real estate magazine called "Al Bab World" in 2007. The platform transitioned to a digital real estate portal, [propertyfinder.ae](https://www.propertyfinder.ae), which quickly became the leading platform in the UAE.

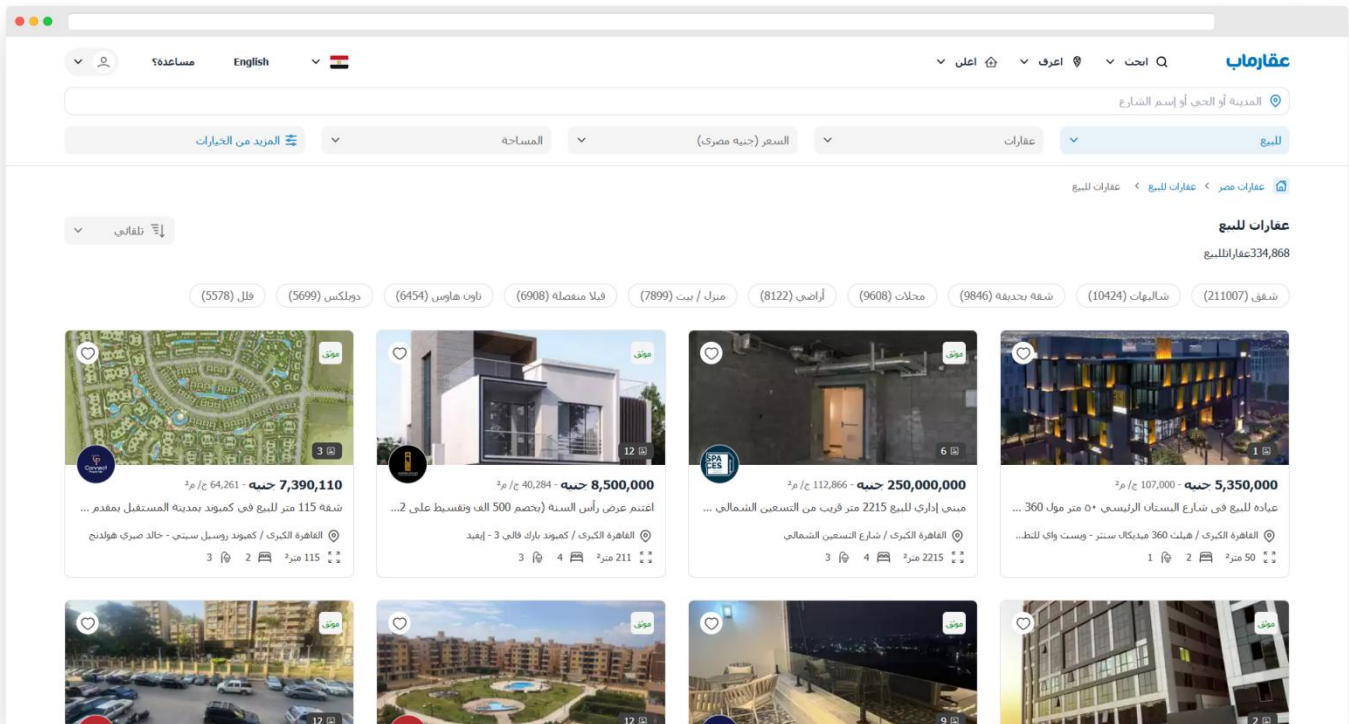
Community Impact

Property Finder emphasizes community growth by offering innovative solutions that enhance the home-buying journey and foster sustainable real estate practices.

Business Model

Property Finder operates under a commission-based model, where it generates revenue by charging real estate agents and developers a fee to list properties on its platform. Additionally, the platform offers premium services for agents, such as "SuperAgent" and marketing tools. The company also monetizes through advertisements and offers data insights for industry professionals. This model enables Property Finder to provide value to both users and real estate professionals while continuously investing in new technology and services.

2.3 Aqarmap (<https://aqarmap.com.eg>)

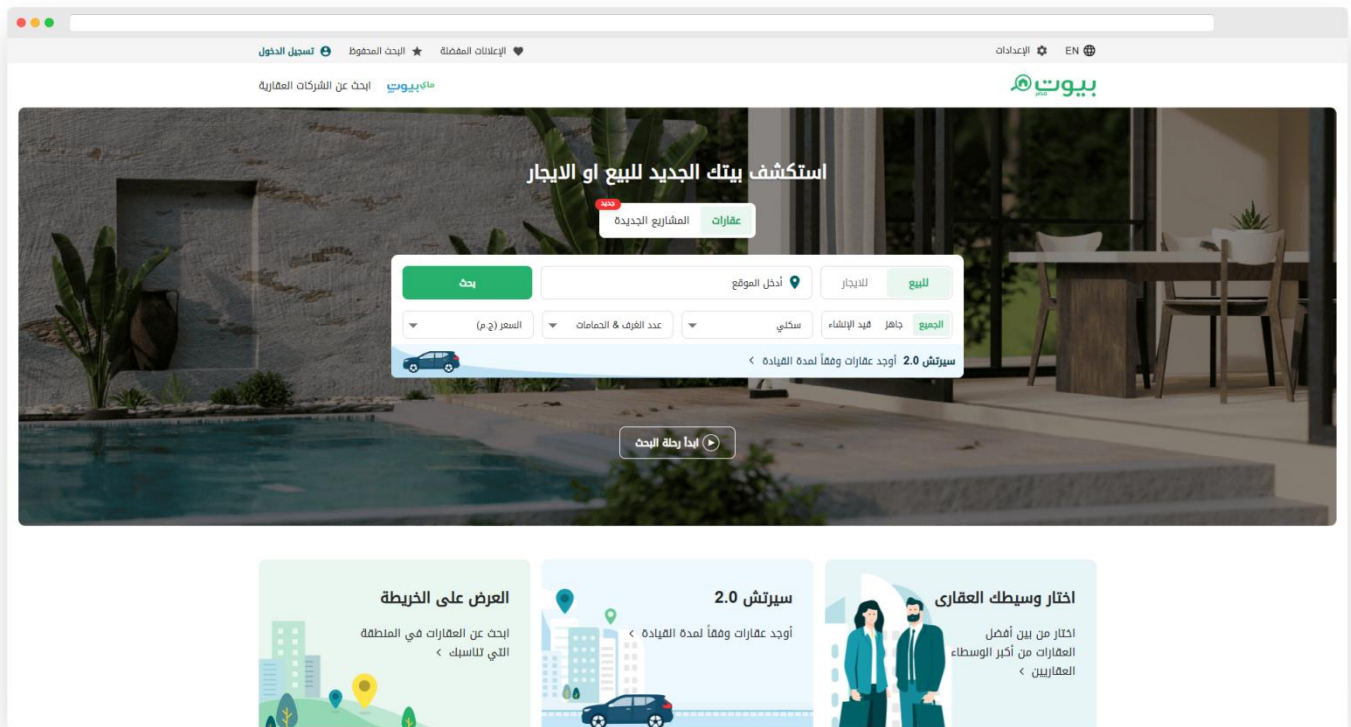


Aqarmap is the first Arabic real estate marketing platform, launched in 2010 by Imad Al-Masoudi. It offers a comprehensive platform for buying, selling, and renting real estate across Egypt, with more than 300,000 properties and over 1,000 compounds listed. The platform serves over 2 million visitors monthly and provides detailed, up-to-date information about properties and developments, helping users make informed decisions.

Business Model

Aqarmap generates revenue through a commission-based model, where real estate developers, property owners, and agents pay to list their properties on the platform. They offer additional paid services such as premium listings, advertising, and market research for real estate companies. The platform also partners with banks and financial institutions to provide users with access to financing options. Furthermore, Aqarmap monetizes through its real estate expos by charging developers and exhibitors to showcase their properties.

2.4 Bayut (https://www.bayut.com)

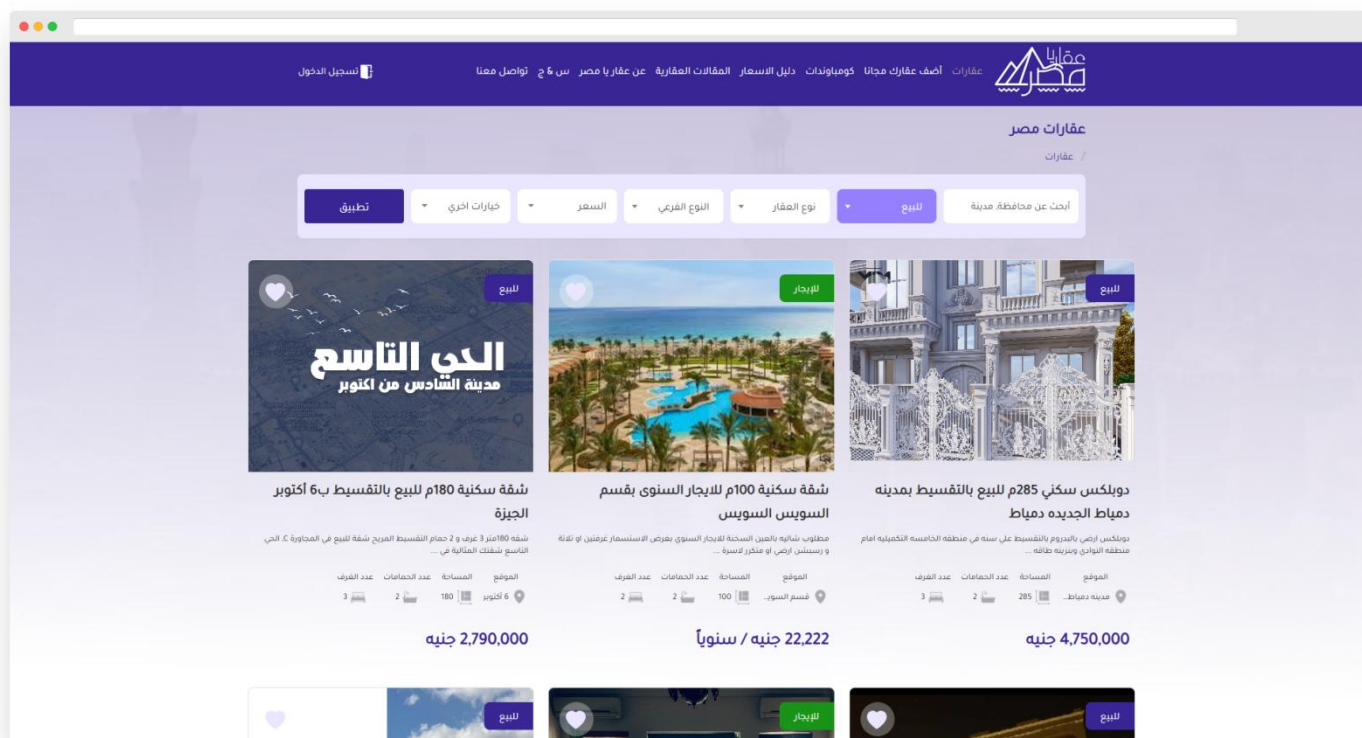


Bayut is a leading property listing platform in the United Arab Emirates, connecting buyers, investors, property owners, sellers, tenants, and agents in a seamless and user-friendly manner. The company is part of Dubizzle Group Holdings Limited, one of the first billion-dollar companies in the UAE, and operates under the motto "Stronger Together" in partnership with Dubizzle, the leading classifieds website in the UAE.

Business Model

Bayut operates on a freemium business model, where property owners and real estate agents can list properties for free or choose to pay for premium listings for greater visibility. The platform generates revenue through advertisements, subscription plans, and featured listings, offering additional exposure and tools for agents and developers. The company's strategic partnerships with investors, developers, and financial institutions further strengthen its presence in the real estate market.

2.5 Aqaryamasr (<https://aqaryamasr.com>)



The real estate market in Egypt continues to expand rapidly, despite the significant urban development happening across the country. However, many Egyptians still face difficulty finding ideal residential properties at reasonable prices, whether for sale or rent.

To solve this problem, we created Aqar Ya Masr—a platform designed to bridge the gap between buyers' needs and what's available in the market, without the involvement of brokers. The platform offers a powerful search engine that allows users to set specific criteria for their ideal property. Once the criteria are entered, the platform quickly filters through listings, displaying only those that match the buyer's specifications. This offers flexibility and freedom of choice, ensuring users can find the best available options.

One of the key features of our advanced search is its ability to provide an overview of current property prices and detailed information on residential, commercial, or administrative units in the selected area. This transparency is crucial for anyone looking to move to a new city, as it helps compare different areas for living, setting up a business, or finding a suitable office, all based on accurate market data.

Chapter 3

Domain Analysis and Technique

3.1 Domain Analysis

3.1.1 Clients and users

1.Users:

Customer Service: is responsible for handling accepting or rejecting requests to add a house from tenants, confirming the inspection, adding the house data after completing the inspection, reviewing all complaints from tenants on the site, and taking the necessary action.

2.Clients:

Tenants: they can use the platform to search for a house suitable for their specifications, interests, and money, rent a house during the appropriate period for them, and cancel the rental request during the period allowed for them to cancel. They must confirm arrival to the house in order to recover their money. They can use the platform to send a complaint in the event of any change in the specifications of the house or they were deceived and can set rate or comments.

Landlords: They are responsible for requesting to add a house, and after completing the inspection, they can review the house, publish the house, update the house's information, or delete the house. They are responsible for accepting or rejecting rental requests from tenants and confirming tenants' arrival to the house.

3.1.2 The Environment

Web-Based Application:

- ◆ **Framework:** Utilizes React JS and Tailwind for the frontend, providing a responsive and interactive user interface.
- ◆ **Backend Operations:** .Net, Sql server for server-side operations, ensuring efficient data processing and management.

Server Infrastructure:

- ◆ **Hosting Services:** Accommodates databases, applications, and essential services crucial for user interaction and data storage
- ◆ **ASP.NET Integration:** Optimized for efficient database interactions and LINQ queries for optimized data access.

Cloud Services and Networking:

- ◆ **Google Translation API:** used to translate dynamic English content to Arabic. (For static content, we use I18Next Library provided by React.)

- ◆ **Firebase Storage:** used to store high-resolution images of rental properties uploaded by landlords.

Development and Testing:

- ◆ **GitHub:** version control system that helps us to apply Agile Development in the application, makes it flexible and easy to update by any team member.
- ◆ **Google Analytics:** for user behavior tracking and Firebase Performance monitoring for real-time insights.
- ◆ **Google Lighthouse:** Web Application Evaluation tool that is used to check the performance, accessibility, and other important properties of the website

3.1.3 Tasks and actions currently being performed

3.2 Risks

Risk Management Plan for Platform Development

Risk	Description	Impact	Mitigation
Regulatory Compliance Failure	Mishandling or improper storage of tenant or landlord data.	Loss of user trust, leading to decreased platform adoption and retention.	Implement a privacy policy outlining data collection, storage, and sharing practices.
Technological Challenges	Compatibility issues between frontend and backend, leading to functionality problems.	System malfunctions, slow performance, and potential downtime affecting user experience	Conduct rigorous testing at each development phase. Allocate additional resources for tech compatibility checks and system integration.
Resource Constraints	Insufficient budget allocation or limited skilled workforce for development and maintenance	Delays in project timelines, compromised quality, and incomplete feature implementation	Prioritize project requirements and allocate resources accordingly. Consider outsourcing for specialized skills if necessary
User Adoption	Low user engagement and participation within the platform's community and forums	Reduced knowledge sharing, lack of collaboration, and limited platform utilization	Implement incentive programs to encourage user contributions. Enhance user experience and interface for better engagement
Data Security and Privacy failure	Breaches in data security or failure to comply with privacy regulations	Compromised user data, legal implications, loss of trust, and damage to reputation	Implement robust encryption methods, regular security audits, and compliance checks. Train staff on data handling protocols
Dependency on Third-Party Services	Reliance on external cloud services or APIs for platform functionalities	Service disruptions, limited control over downtime, and potential data loss	Diversify service providers, ensure contingency plans for service outages, and regular data backups
Platform Downtime	Unplanned interruptions in platform accessibility due to technical issues or maintenance	Disruptions could hinder user experience and engagement temporarily	Regularly schedule maintenance during off-peak hours. Implement robust backup and recovery mechanisms to minimize downtime
Temporary Staff Shortage	Short-term unavailability of specific skilled staff due to unforeseen circumstances (e.g., sickness, temporary leave).	Could cause minor delays or shifts in responsibilities.	Cross-train team members, maintain clear documentation of responsibilities, and have contingency plans for temporary staff shortages

Risk Assessment Matrix

Likelihood	Impact				
	INSIGNIFICANT	MINOR	SIGNIFICANT	MAJOR	SEVERE
	LIKELY	Temporary Staff Shortage	User Adoption	Technological Challenges	
	POSSIBLE		Resource Constraints		Regulatory Compliance Failure
	UNLIKELY		Dependency on Third-Party Services	Platform Downtime	Data Security and Privacy failure

3.3 Constrains

Regulatory compliance

consent management: Explicitly obtain user consent for data collection and inform users about how their data will be used.

short-term rental laws: Adhere to city- or country-specific laws governing short-term rentals, such as maximum stay durations or landlord registration requirements.

Technological limitations

high traffic and load management: Handling increased user traffic, especially during peak rental seasons or for popular listings, might strain the system. Scaling a .net and react-based system dynamically to accommodate fluctuating demand can be challenging.

machine learning latency: Real-time pricing recommendations and market analysis powered by python-based machine learning could face delays, especially if large datasets are involved or the server lacks sufficient computational power.

model maintenance: Constantly updating models to reflect changing market conditions is resource-intensive and requires skilled personnel.

Resource constraints

human resources skilled personnel: Developing and maintaining a platform using .net, react, and python (for machine learning) requires expertise in multiple programming languages and frameworks, which may be hard to source or expensive.

operational costs: Hosting, scaling infrastructure, and integrating third-party apis (e.g., id verification, payment gateways) can be expensive.

Time constraints

integration delays: Time-consuming processes for integrating third-party apis and ensuring compatibility across different technologies.

iterative testing: Thorough testing for security, performance, and user experience often takes longer than anticipated, delaying deployment.

Security considerations

breach prevention: Robust cybersecurity measures are essential to prevent unauthorized access, data theft, or ransomware attacks.

review manipulation: Preventing false reviews or ratings that could undermine trust.

open-source dependencies: Ensuring all libraries and frameworks used are up-to-date and free from known vulnerabilities.

User adoption and engagement

intuitive design: A clean, responsive, and user-friendly interface that guides users smoothly through the rental process.

streamlined workflows: Simple search, filter, and rental processes that require minimal effort from users.

Device compatibility: Optimized performance on both desktop and mobile devices.

3.4 Project plan

3.5 Feasibility Study

3.5.1 Technical feasibility

SECTION	DESCRIPTION
Software Requirements	The platform will utilize React.js and Tailwind for the frontend, .net for the backend, Sql server as the database system and Python for machine learning integrations. The system should run on servers with Window OS, supporting the latest versions of .net, Sql server and Python.
Hardware Requirements	The infrastructure requirements include scalable cloud hosting services like Firebase storage.
Technology Assessment	React.js and .net are selected for their robustness, scalability, and extensive community support, enabling rapid development and a responsive user interface. Sql server is chosen due to its flexibility in handling structured data and its scalability for accommodating future data growth. Python for machine learning integrations and simplicity.
Technical Expertise	We have a team of skilled developers experienced in React.js, .net, Sql server, and python minimizing the need for additional recruitment or training.
Development Tools	Leveraging development tools like Visual Studio Code, Git for version control, and Visual Studio will facilitate efficient development workflows.
Security Measures	Implementing SSL encryption and regular security patches will safeguard user data, meeting industry standards and compliance regulations.

3.5.2 Operational feasibility

SECTION	DESCRIPTION
User Acceptance Rate	Foresee a high user acceptance rate of 90% based on the platform's intuitive interface and user-friendly functionalities, allowing easy navigation and utilization for tenants, landlords, and customer service.
System Integration Period	Estimate a 5-month integration timeline to ensure seamless adoption and utilization of Symsary's diverse functionalities.
Throughput Enhancement	Project a 40% improvement in operational efficiency, expecting improving the system's ability to process a higher volume of requests, transactions, or operations efficiently within a given time.
Information Accessibility	Anticipate a 50% increase in timely access, provide real-time information on availability, pricing, and rental status to ensure accuracy. Use notifications for important updates like rental confirmations or policy changes.
Adaptation Period	Expect a 2-month adaptation period for users to acclimate to Symsary's unique features, leveraging interactive guides and forums to facilitate a smooth transition from conventional practices.
User Satisfaction	Aim for a 91% satisfaction rate among users post implementation, focusing on user-centric design and continual feedback incorporation to enhance the platform's usability.

3.5.3 Economic feasibility

SECTION	DESCRIPTION	COST (EGP)
Development Costs	Software Development: Anticipate initial software development costs based on consultations with development teams and market analysis.	60,000
	Infrastructure Setup: Estimate infrastructure setup costs including server hosting, database setup, and system integration.	40,000
	Resource Allocation: Allocate costs for skilled resources, project management, and training during the development phase.	20,000
Operational Costs (Annual)	Maintenance: Estimate annual maintenance costs considering software updates, bug fixes, and server maintenance.	15,000
	Staff salary: Estimate staff salary costs for customer service agents for each time to review houses annually.	40,000
	Hosting: Hosting expense for server hosting and cloud services.	15,000

Initial Investment: 120,000 EGP

Project duration for analysis: 5 years

SECTION	DESCRIPTION
Cost Savings	Expect cost savings of approximately 20,000EGP annually through the increase of houses already available and confirmed on the platform, which not need to inspection.
Revenue Projections	Reviews Service: 20,000 EGP annually (Revenue generated from reviews of houses before being added to the platform)
	House Advertisement adding: 60,000 EGP annually (Revenue generated from landlords' subscription to different plans of advertising on platform)

Year	Cost Savings	Reviews Service	House Advertisement	Total Benefit	Total Costs	Net Cash Flow	Cumulative Net Cash Flow	ROI
0	-	-	-	-	120,000	-120,000	-120,000	- 100%
1	20,000	20,000	60,000	100,000	70,000	30,000	-90,000	25.0%
2	20,000	20,000	60,000	100,000	70,000	30,000	-60,000	25.0%
3	20,000	20,000	60,000	100,000	70,000	30,000	-30,000	25.0%
4	20,000	20,000	60,000	100,000	70,000	30,000	0,000	25.0%
5	20,000	20,000	60,000	100,000	70,000	30,000	30,000	25.0%

The payback period is approximately between in the fifth year, where the cumulative net cash flow turns positive and exceeds the initial investment of 120,000 EGP. Therefore, the estimated payback period for this investment is about 4 years.

3.6 Quality Assurance Plan

Automated testing :

In the Platform the AT is a cornerstone of Semsary's QA process, focusing on unit and integration testing. Unit testing involves the creation of detailed tests for individual components, such as functions and methods, which are executed in isolation to maintain code integrity. Developers write detailed tests for individual components, functions, and methods in isolation. These tests are executed automatically with each code modification, ensuring changes do not disrupt existing functionality. Any code changes trigger automated execution of these tests, quickly identifying and resolving potential bugs. Integration testing, on the other hand, validates the interactions between various modules and components, ensuring smooth communication and proper functionality across the system. Tools like Jest and Postman are employed to verify the compatibility and behavior of integrated components. Automated tests are seamlessly integrated into the platform's CI/CD pipelines, allowing each code commit or deployment to trigger immediate testing, providing early detection of any regressions or inconsistencies. Developers receive immediate alerts about any failures.

Manual testing :

Manual testing complements automated testing by focusing on exploratory and usability testing. Exploratory testing allows testers to navigate the platform organically, simulating real-world scenarios to uncover hidden defects and unexpected behaviors. This unscripted approach ensures that potential issues missed by automated tests are identified and addressed. Usability testing evaluates the platform's interface, navigation, and user-friendliness by simulating user journeys. Feedback collected during these tests is used to refine the platform's design, aligning it more closely with user expectations and improving the overall user experience.

Regression testing :

It is critical to maintaining the stability and reliability of the Semsary platform. This phase ensures that recent updates or changes do not adversely affect the platform's existing functionality. A selective suite of critical test cases, focusing on high-impact areas such as the booking system, user authentication, and payment processing, is executed automatically after each deployment. These regression tests are integrated into the CI/CD pipeline, with results compared against baseline outcomes to detect any deviations or inconsistencies. By prioritizing high-risk scenarios, the platform ensures continuous functionality while accommodating changes.

Performance testing :

It ensures that the Semsary platform is scalable, robust, and reliable under various load conditions. Load testing simulates expected user behavior to evaluate the platform's response times, resource utilization, and overall performance during regular operations. Automated tools are used to monitor key metrics like throughput and latency. Stress testing complements this by subjecting the system to extreme loads, identifying its breaking points, and observing recovery behavior after stress conditions are removed. This approach ensures the platform remains stable and performs optimally even during high-demand scenarios.

Security testing :

It is a critical aspect of the QA process, focusing on protecting the platform against vulnerabilities and ensuring data confidentiality. Vulnerability scanning is conducted using automated tools to systematically identify weaknesses, such as misconfigurations or outdated components, which are then prioritized and mitigated. Additionally, penetration testing simulates real-world cyberattacks to identify potential entry points for unauthorized access. This testing, performed by skilled security professionals, assesses the resilience of authentication mechanisms, payment systems, and data storage. By addressing identified threats promptly, Semsary ensures a secure environment for its users.

This structured quality assurance plan ensures that Semsary consistently delivers a reliable, secure, and user-friendly experience, aligning with its mission to provide transparency, trust, and convenience for its users.

3.7 System Requirements

3.7.1 Functional Requirements

Tenants:

1. Account Management:

- FR-1: The system should enable tenants to create an account and get verification via email.
- FR-2: The system should allow tenants to securely login using an email and password, and reset their login credentials if forgotten.
- FR-3: Tenants should be able to edit their profile information.

2. Search and Booking:

- FR-4: The system should allow tenants to search for properties and apply filters to find suitable listings.
- FR-5: Tenants should be able to send a booking request to a landlord if they have enough balance on the website.

3. Communication:

- FR-6: The system should enable tenants and landlords to communicate through an integrated chat feature.

4. Financial Transactions:

- FR-7: The system should allow tenants to deposit and withdraw money using credit cards or electronic wallets.
- FR-8: Tenants should be able to get back their money if they canceled the renting request during 4 days from date of request but after this period, the money will be transferred to landlord balance.
- FR-9: Tenants should be able to confirm their arriving to house to get their warranty money back.

5. Feedback and preview:

- FR-10: System should enable tenants to request for a house preview in case the house specifications do not match the advertisement.
- FR-11: Tenants should have the ability to give a rate of houses that they had already rented.

Landlord:

1. Account Management:

- FR-12: The system should allow landlords to create an account and get verification via SMS.

- FR-13: The system should enable landlords to securely login using their phone number and password and reset their credentials if forgotten.
 - FR-14: Landlords should be able to update their profile information.
2. House Advertisement Management:
 - FR-15: The system should allow landlords to request an On-the-ground preview for their houses.
 - FR-16: The system should provide landlords access to all houses advertisements that have been inserted into their profile after preview.
 - FR-17: Landlords should be able to update specific details in their houses advertisements.
 - FR-18: Landlords should be able to publish their houses if they have enough balance and have the ability to delete them.
 3. Financial Transactions:
 - FR-19: The system should allow landlords to deposit and withdraw money using credit cards or electronic wallets.
 4. Rental Management:
 - FR-20: Landlords should be able to review, accept, or reject rental requests from tenants.
 - FR-21: The system should enable landlords to confirm tenants arrival requests to enable tenants get their warranty money back.

Customer Service:

1. Account Management:
 - FR-22: The system should allow customer service to securely login using an email and password, and reset their login credentials if forgotten.
 - FR-23: customer service should be able to edit their profile information.
2. Reviews Management:
 - FR-24: Customer service should be able to receive landlords requests for reviews and enter the house advertisement information.
 - FR-25: Customer service should be able to receive tenants' reviews requests.

3.7.2 Non-Functional Requirements

1. Performance Requirements:
 - NFR-1: The system should be able to handle up to 7,000 simultaneous user requests without performance degradation.
 - NFR-2: Page load time should not exceed 3 seconds under normal load conditions.
 - NFR-3: The platform should process payment transactions in under 5 seconds.
2. Scalability Requirements:
 - NFR-4: The system should scale dynamically to accommodate an increase in users or data volume without performance degradation.
3. Availability Requirements:
 - NFR-5: The platform should have an uptime of at least 98% annually.

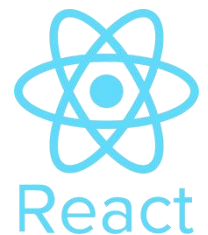
- NFR-6: The system should provide redundancy mechanisms to minimize downtime during maintenance or unexpected failures.
4. Security Requirements:
 - NFR-7: All user data must be encrypted during transmission using HTTPS and TLS protocols.
 - NFR-8: Passwords should be hashed and stored securely using an industry-standard hashing algorithm.
 - NFR-9: The system must be safeguarded against common web vulnerabilities like XSS, SQL injection, etc.
 5. Usability Requirements:
 - NFR-10: The user interface should follow accessibility standards and be user-friendly.
 6. Reliability Requirements:
 - NFR-11: The system should recover from critical failures within 10 minutes.

3.8 Techniques and tools

3.8.1 Frontend Framework

1.React JS:

React.js was selected as the core framework for developing the frontend of the apartment rental platform due to its modular and dynamic nature. Key features used include:



Component-Based Architecture:

Built reusable components such as property cards, search bars, filters, and user authentication modals, Ensured consistency and reduced redundancy by organizing components logically within feature-based folders.

React Hooks:

Used useState to manage states like filter selections, form inputs, and user authentication, Implemented useEffect for handling side effects, such as fetching property data from Firebase in real-time and monitoring changes.

React Router:

Integrated React Router to enable navigation across pages (e.g., Home, Apartment Listings, User Profile, and About Us) without full page reloads, creating a seamless user experience

2. Styling Framework:



Tailwind CSS:

Tailwind CSS was used for its efficiency in designing modern, responsive, and visually appealing interfaces tailored for real estate listings.

Custom Themes:

Configured Tailwind's theme in `tailwind.config.js` to reflect a professional color palette, such as warm neutrals and blues, representing trust and reliability in real estate.

Responsive Design:

Ensured the website was mobile-friendly by designing layouts with responsive utility classes to accommodate all screen sizes, from smartphones to large monitors.

3. State Management



To manage data flow and application state:

React Context API:

Used for sharing global states like active filters (e.g., price range, location, and amenities) and user login status across the application without prop drilling.

Local State Management: Managed component-specific states for dynamic features like search queries and form validations.

Redux

Redux was used to handle the global state of the apartment rental platform efficiently. Key features include:

Centralized State: Managed filters, apartment listings, user authentication, and reviews in a single global store.

Actions and Reducers: Defined reusable actions (e.g., `FILTER_APARTMENTS`, `ADD_REVIEW`) and reducers to update the state predictably.

Middleware: Used `redux-thunk` for asynchronous tasks like fetching apartment data and submitting reviews.

React Integration: Connected Redux to React using `react-redux`, utilizing `useSelector` for accessing state and `useDispatch` for dispatching actions.

Improved Performance: Cached data like apartment listings to minimize API calls and enhance responsiveness.

Build Tools: Vite and npm

Vite: Used for fast development with Hot Module Replacement (HMR) and optimized production builds. Custom configurations in vite.config.js supported Tailwind CSS and Firebase integration.



Vite

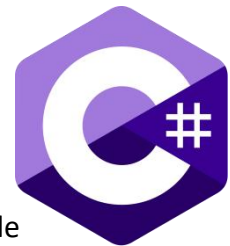
npm: Managed dependencies (e.g., React, Redux, Firebase) and provided scripts for development (npm run dev), production build (npm run build), and preview (npm run preview).



3.8.2 Backend Development

1.C#

C# was chosen as the programming language for backend development due to its versatility and strong integration with .NET. Key features include:



Object-Oriented Programming: Enabled modular, reusable, and maintainable code, essential for implementing business logic for user management, apartment listings, and reviews.

Asynchronous Programming: Used async/await for handling asynchronous tasks like database queries and API calls, improving server responsiveness.

Robust Error Handling: Implemented try-catch blocks and custom exception handling to ensure stability and provide detailed error messages for debugging.

Integration Support: C#'s rich library ecosystem facilitated seamless integration with Firebase for image storage and third-party tools like Google Analytics.

2.NET Framework

ASP.NET Core, a modern and high-performance web framework, powered the backend with the following features:



RESTful API Development: Designed endpoints for CRUD operations on apartments, user accounts, and reviews. Supported HTTP methods like GET (retrieve apartments), POST (add reviews), PUT (update user info), and DELETE (remove listings).

Middleware: Used custom middleware for logging, error handling, and request validation. Implemented authentication middleware to validate tokens and secure API access.

Dependency Injection:

Integrated DI to manage services like database contexts and external API clients, enhancing code modularity and testability.

Authentication and Authorization:

Implemented JWT (JSON Web Token) for user authentication and role-based authorization to differentiate access for normal users, premium users, and admins.

1. SQL Server

SQL Server served as the relational database for managing structured data in the application. Key aspects include:



Database Design:

Tables were designed with proper normalization to reduce redundancy and improve data integrity.

Users: Stored user details, authentication information, and roles.

Apartments: Included details like location, price, amenities, and image URLs.

Reviews: Managed user feedback with fields for ratings, comments, and timestamps.

Stored Procedures:

Created stored procedures for complex queries like fetching apartments with filters applied and calculating average ratings.

Improved performance by reducing the load on the application layer and optimizing query execution.

Indexes:

Added indexes on frequently queried fields (e.g., apartment location, price) to speed up search operations.

Transactions:

Used transactions to ensure consistency during critical operations, such as booking apartments or submitting reviews, to prevent partial updates.

3.8.3 Machine Learning Model

1. Python

Python served as the primary programming language for implementing the



recommendation system due to its:

Rich Ecosystem: Extensive libraries and tools for data manipulation, visualization, and machine learning.

Ease of Use: Simple syntax that accelerates development and experimentation.

Community Support: Access to a vast range of resources, tutorials, and pre-built models.

2. TensorFlow

TensorFlow, an open-source machine learning framework, was used for building and training the recommendation model.



Key Features Utilized:

Deep Learning Support: Designed a neural network architecture optimized for collaborative filtering or content-based recommendations.

Ease of Deployment: TensorFlow's model export capabilities allowed easy integration of the trained model with the backend.

TensorFlow Recommenders (TFRS): A specialized library within TensorFlow, TFRS provided prebuilt tools for creating recommendation systems, reducing development time.

Advantages:

Scalability: Handled large datasets of user reviews, apartment details, and preferences efficiently.

Performance: TensorFlow's GPU support accelerated training times for large-scale models.

Customizability: Allowed fine-tuning of the model to cater to unique aspects of the rental platform, such as prioritizing recent reviews or higher-rated apartments.

3.8.4 Third Party Services

1. Firebase Storage

Firebase Storage was utilized for storing and managing user-uploaded images, such as apartment photos. Its integration simplified handling large files and ensured reliable, scalable storage. Key features include:

Seamless Integration:

Integrated with the Firebase SDK to easily upload, retrieve, and manage image files directly from the application.



Scalability:

Supported storing high-resolution images without impacting app performance, ensuring the platform can handle increasing user activity.

Security Rules:

Configured Firebase security rules to control access, ensuring only authenticated users can upload images and restricting downloads as per user roles.

Real-Time URL Generation:

Generated secure download URLs after each upload, which were stored in the database and used to display images on the platform.

Cost-Effectiveness:

Benefited from Firebase's pay-as-you-go pricing, making it a cost-efficient choice for storage needs.

2. SendGrid:**Email Delivery Service**

SendGrid is a cloud-based email delivery platform that provides businesses with the tools to send transactional and marketing emails at scale. It offers a range of APIs and SMTP services for email delivery, along with analytics to track email performance, ensuring high deliverability rates and reliable service. SendGrid is popular among developers for its ease of integration with both small applications and large-scale enterprise systems.



Email API: Allows developers to send email messages from their applications, with options for both transactional and marketing emails.

SMTP Relay: Provides a reliable service for sending emails using the SMTP protocol, making it easy to integrate into existing systems.

Deliverability Tools: Includes tools to track email delivery, open rates, click rates, and more, to optimize performance.

Templates and Personalization: Users can create and send customized email templates for better engagement with recipients.

Scalability: Supports both small and large-scale email sending needs, suitable for businesses of any size.

Security: Offers features like email authentication and encryption to protect against phishing and unauthorized access.

Use Cases:

Sending transactional emails (e.g., order confirmations, password resets).

Running marketing campaigns with advanced email personalization.

Integrating email services with web applications via API.

3. PayMob: Payment Gateway



PayMob is a leading payment gateway in Egypt, designed to facilitate secure online transactions for businesses and consumers. It provides an API for processing payments, enabling businesses to accept payments via credit/debit cards, mobile wallets, and other local payment methods. PayMob is widely used by e-commerce businesses, subscription services, and retail merchants to offer a seamless and secure payment experience.

Key Features:

Payment Gateway API: Allows businesses to process payments from customers via a wide range of local and international payment methods.

Mobile Wallet Integration: Supports popular mobile wallets, including Fawry, Vodafone Cash, and others for easy mobile-based payments.

Recurring Payments: Enables subscription-based services to collect payments on a regular basis.

Fraud Detection: Provides advanced security features, including fraud prevention tools, to protect businesses from fraudulent transactions.

Invoicing: Facilitates generating and managing invoices directly from the PayMob platform.

Local Integration: Tailored specifically for the Egyptian market, with support for local banks and payment methods.

Use Cases:

Enabling e-commerce sites to accept payments securely.

Providing subscription-based businesses with recurring billing solutions.

Integrating mobile wallet payments for greater convenience.

4. Twilio: SMS Messaging API

Overview:

Twilio provides a powerful SMS API for businesses to send and receive text messages globally. It allows seamless integration of SMS capabilities into web and mobile applications, enabling businesses to automate notifications, alerts, reminders, and two-way messaging. With robust scalability and reliable delivery, Twilio ensures effective communication for organizations of all sizes.



Key Features:

Send SMS:

Quickly send text messages to customers worldwide through a simple API.

Receive SMS:

Enable two-way communication by receiving replies directly in your application.

Phone Number Management:

Purchase, configure, and manage virtual phone numbers to customize messaging.

Global Reach:

Deliver SMS to over 180 countries with high reliability and fast delivery rates.

Programmable Messaging:

Personalize messages with templates and variables for tailored communication.

Delivery Status Tracking:

Monitor the status of sent messages with detailed delivery reports.

Scalability:

Supports large-scale messaging for campaigns, promotions, or transactional alerts.

Use Cases:

Sending transactional messages like OTPs, confirmations, and alerts.

Running SMS-based marketing campaigns.

Automating customer notifications, reminders, and updates.

5. Google Analytics

Google Analytics was integrated into the project to track and analyze user interactions, providing valuable insights for improving the platform.



Key Features:

Real-Time Monitoring:

Tracked active users, pages visited, and interactions as they occurred.

User Behavior Analysis:

Collected data on how users navigated the site, including frequently visited pages, filters used, and time spent on listings.

Traffic Sources:

Identified where users originated from (e.g., search engines, social media, or referrals), helping optimize marketing strategies.

Event Tracking: Monitored specific actions, such as apartment searches, clicks on "Contact Owner," and image uploads.

Helped understand user preferences and behavior, enabling data-driven improvements to the platform.

Monitored conversion rates, such as users signing up or submitting apartment reviews.

6. Version Control: Git and GitHub

Git: Used locally for tracking code changes, managing feature branches and enabling easy rollbacks.



GitHub: Hosted the repository for collaboration, pull requests, and code reviews, while serving as a secure backup.

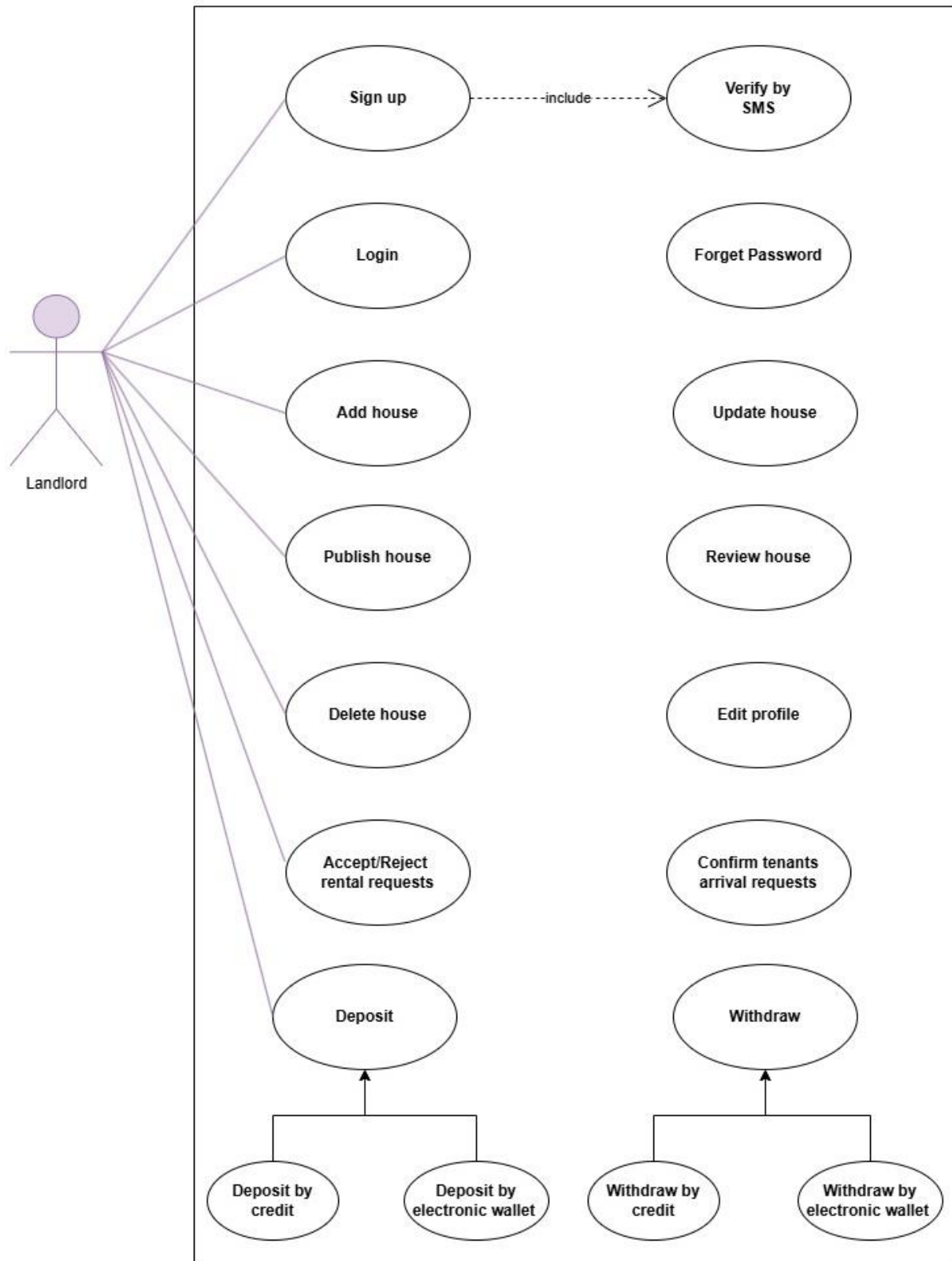


Chapter 4

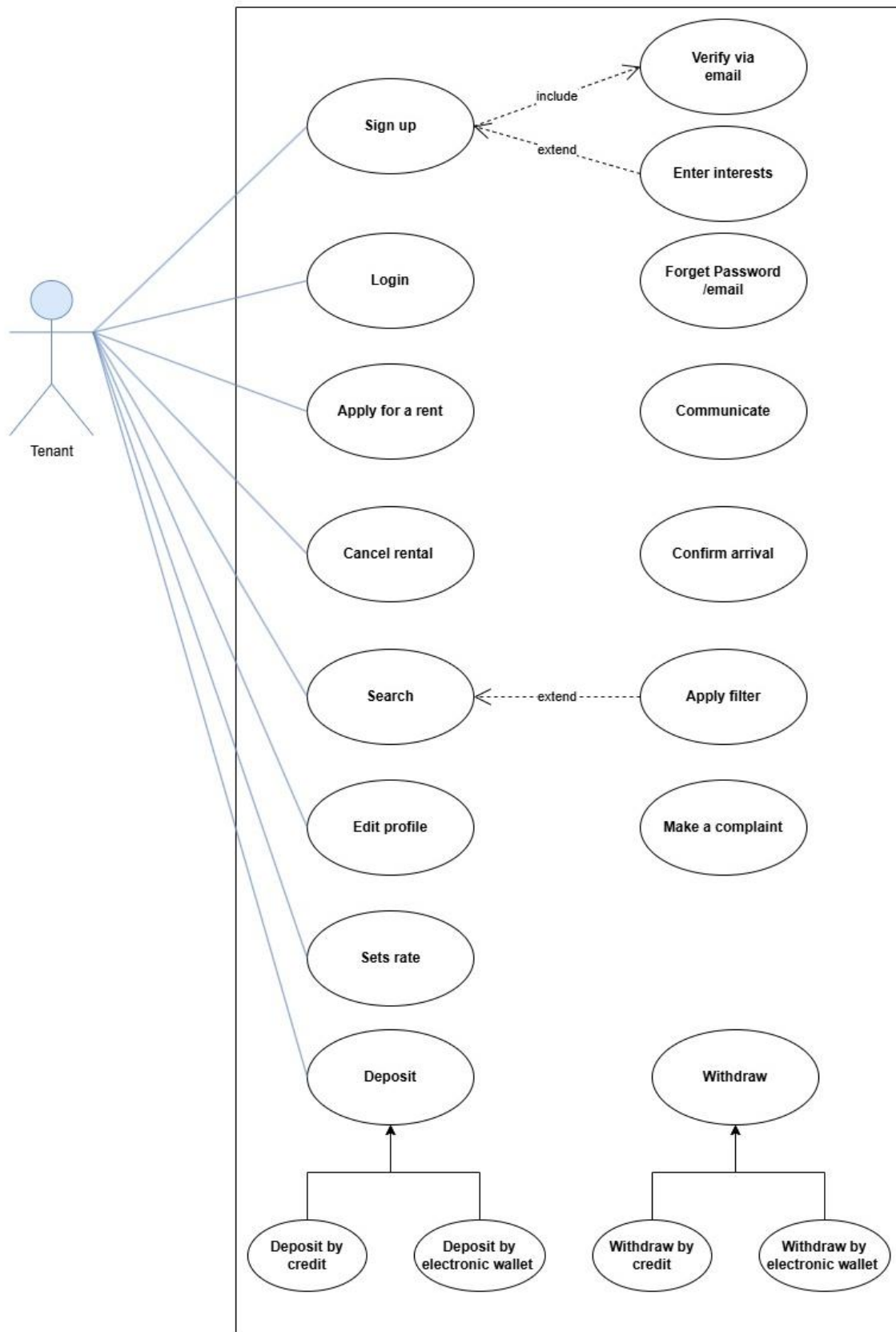
Proposed System & Methodology

4.1 System Use-Cases

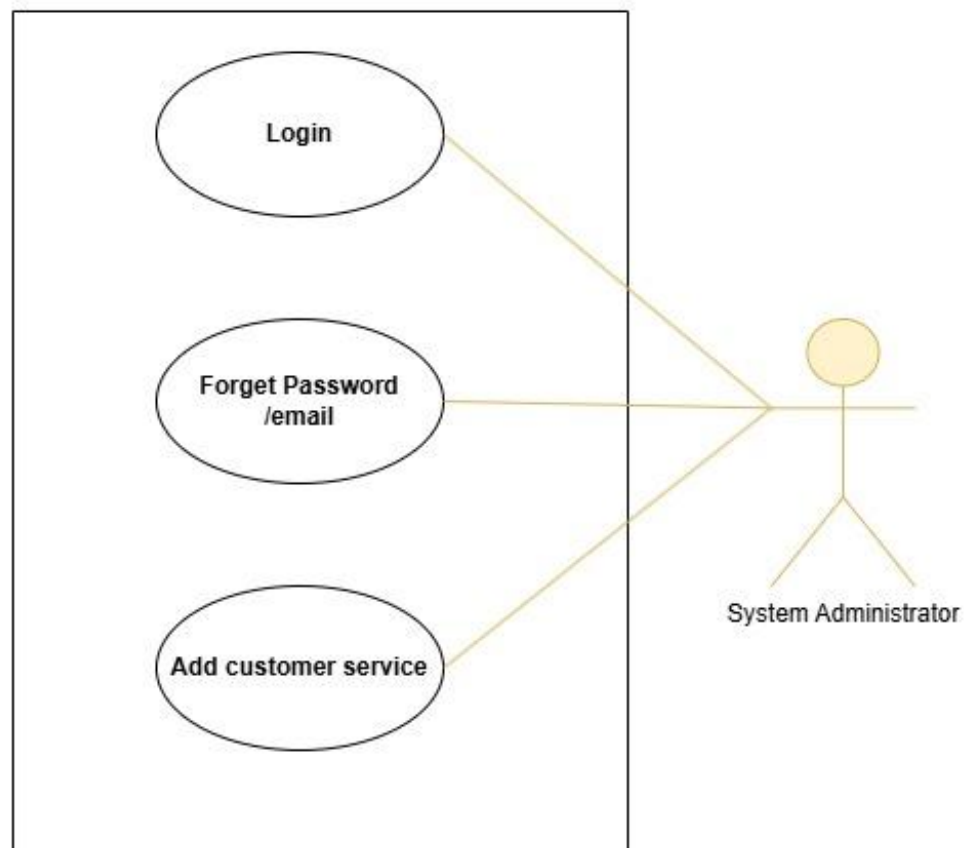
4.1.1 Landlord



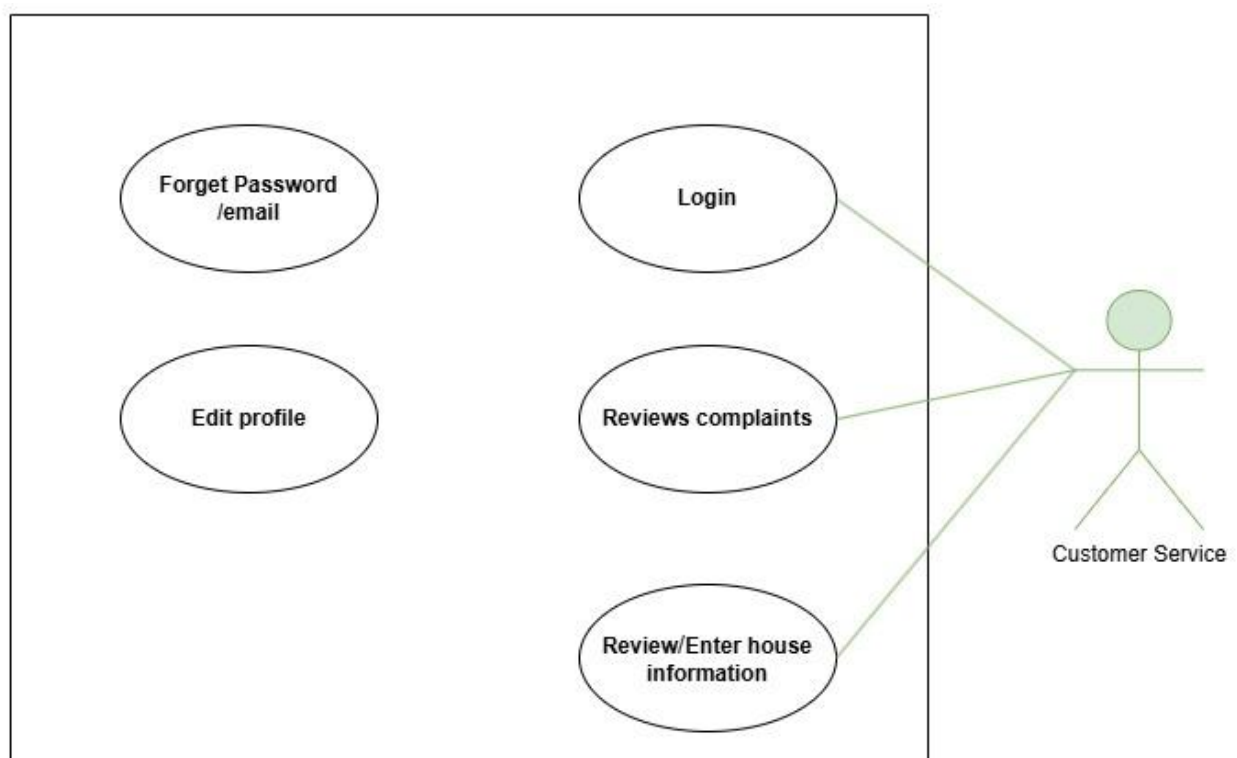
4.1.2 Tenant



4.1.3 Admin Use-Case



4.1.4 Customer Service



4.2 Use Case Description (Use case scenario)

Case Name:	Landlord Sign Up
Case ID	LND-01
Actor(s):	Landlord
Description:	Registers a new landlord on the platform.
Triggering Event:	Landlord selects "Sign Up" option on the platform.
Preconditions:	Landlord is not yet registered. Landlord has required information (phone number, password, proof of identity).
Assumptions:	The platform is accessible and operational. Identity verification is automated and supported by a reliable third-party service.
Main Path:	Landlord enters phone number, password, and uploads identity document. Platform validates the information. If all data is valid, the system creates a new account for the landlord.
Alternate Path:	If the identity document is invalid, the system notifies the landlord and prompts them to upload a valid document.
Postcondition:	New landlord is registered and can access their account.
Priority:	High

Case Name:	Add House
Case ID	LND-02
Actor(s):	Landlord
Description:	Allows landlords to submit a request for customer service to inspect the house.
Triggering Event:	Landlord selects "Add House."
Preconditions:	Landlord is logged in. Landlord has a valid proof of ownership document.
Assumptions:	Customer service agents are available for property inspections. Inspection scheduling occurs within an acceptable timeframe (e.g., 48 hours).
Main Path:	Landlord enters property address and uploads proof of ownership. Platform sends request to customer service for inspection. Customer service inspects property and defines specifications. Platform adds the house to landlord list of advertisements.
Alternate Path:	If proof of ownership is invalid, the system requests resubmission.
Postcondition:	Property is listed for rent on the platform.
Priority:	High

Case Name:	Update House
Case ID	LND-03
Actor(s):	Landlord
Description:	Allows landlord to update availability and pricing information for an existing listing.
Triggering Event:	Landlord selects "Update House" on their listing.
Preconditions:	Landlord is logged in. Landlord has enough balance for publishing the advertisement.
Assumptions:	The platform updates are instantaneous and reflect immediately on listings. Landlord has accurate, up-to-date information about availability and pricing.
Main Path:	Landlord updates availability status and/or price. Platform saves the changes to the listing.
Alternate Path:	If the listing is inactive, the platform notifies the landlord.
Postcondition:	Updated listing is displayed on the platform.
Priority:	Medium

Case Name:	Publish House
Case ID	LND-04
Actor(s):	Landlord
Description:	Allows landlord to publish house from their houses' advertisements list.
Triggering Event:	Landlord selects "Publish House" for advertisement he wants to publish.
Preconditions:	Landlord is logged in.
Assumptions:	Landlord already has houses in his list. Landlord has enough balance.
Main Path:	Landlord chooses the house he want to publish from his list. Landlord choose a suitable plan for the advertisement. The cost of the advertisement will be withdrawn from landlord balance.
Alternate Path:	If the landlord does not have enough balance he will be notified.
Postcondition:	Display the Advertisement on the platform.
Priority:	High

Case Name:	Review House
Case ID	LND-05
Actor(s):	Landlord
Description:	Allows landlord to check houses inserted in his list.
Triggering Event:	Landlord selects "Review House".
Preconditions:	Landlord is logged in.
Assumptions:	Landlord already has houses in his list.
Main Path:	Landlord enters the list of houses that has been inspected and added to his list. Landlord is able to choose any advertisement to check it details.
Alternate Path:	If the landlord does not have houses in his list he will be notified.
Postcondition:	Display the list of Advertisements for landlord.
Priority:	High

Case Name:	Delete House
Case ID	LND-06
Actor(s):	Landlord
Description:	Allows landlord to Delete house advertisement from his list.
Triggering Event:	Landlord selects "Delete House".
Preconditions:	Landlord is logged in.
Assumptions:	There is already advertisement to be deleted.
Main Path:	Landlord chooses any advertisement to delete. Platform saves the changes to the listing.
Alternate Path:	If the landlord does not have houses in his list he will be notified.
Postcondition:	Advertisement will be removed from landlord list.
Priority:	Medium

Case Name:	Accept or Reject Rental Requests
Case ID	LND-07
Actor(s):	Landlord
Description:	Landlord reviews and approves or declines rental requests.
Triggering Event:	Landlord receives a rental request.
Preconditions:	Landlord has an active rental listing. Rental request exists with specified dates.
Assumptions:	Platform messaging system facilitates prompt communication between landlord and tenant. Tenant has sufficient funds for the guarantee if the request is accepted.
Main Path:	Landlord reviews rental request details. Landlord either accepts or rejects the request. Platform updates the request status and notifies the tenant.
Alternate Path:	If the rental dates are unavailable, platform displays an error message.
Postcondition:	Request status is updated, and tenant is notified.
Priority:	High

Case Name:	Confirm Tenant's Arrival
Case ID	LND-08
Actor(s):	Landlord
Description:	Confirms tenant arrival at the rental property.
Triggering Event:	Tenant informs platform of their arrival.
Preconditions:	Rental request is approved, and tenant is scheduled to arrive.
Assumptions:	Platform reliably tracks tenant arrival and departure dates. Landlord can access and confirm tenant arrival without delay.
Main Path:	Landlord receives arrival notification. Landlord confirms tenant's arrival on the platform.
Alternate Path:	If tenant does not arrive within a set timeframe, platform marks request as "Cancelled."
Postcondition:	Rental officially begins, and system updates status.
Priority:	High

Case Name:	Tenant Sign Up
Case ID	TEN-01
Actor(s):	Tenant
Description:	Registers a new tenant on the platform.
Triggering Event:	Tenant selects "Sign Up" on the platform.
Preconditions:	Tenant has an email, password, and proof of identity.
Assumptions:	The platform's identity verification process is swift and reliable. Tenant's device supports document upload functionality.
Main Path:	Tenant enters email, password, and uploads identity document. Platform verifies the information. Account creation is completed if data is valid.
Alternate Path:	If identity verification fails, tenant is prompted to resubmit valid proof.
Postcondition:	New tenant account is created.
Priority:	High

Case Name:	Search for Suitable House
Case ID	TEN-02
Actor(s):	Tenant
Description:	Tenant searches for available houses based on preferences.
Triggering Event:	Tenant accesses search functionality.
Preconditions:	Tenant is logged in.
Assumptions:	Search filters function accurately and are up-to-date. Platform has enough rental listings to provide relevant results.
Main Path:	Tenant selects search filters (e.g., location, price range, number of bedrooms). Platform displays listings that match the criteria.
Alternate Path:	If no listings match, platform suggests alternative options.
Postcondition:	Tenant views a list of suitable properties.
Priority:	Medium

Case Name:	Apply for Rent
Case ID	TEN-03
Actor(s):	Tenant
Description:	Tenant submits a rental request to the landlord.
Triggering Event:	Tenant selects "Apply for Rent."
Preconditions:	Tenant is logged in. Tenant has sufficient funds for the guarantee amount.
Assumptions:	Guarantee amount covers initial rental period and any fees. Landlord responds within a reasonable time to tenant requests.
Main Path:	Platform forwards request to landlord and updates status. The warranty money required is withdrawn from landlord balance.
Alternate Path:	If funds are insufficient, platform prompts for deposit.
Postcondition:	Rental request is created, and landlord is notified.
Priority:	High

Case Name:	Communicate
Case ID	TEN-04
Actor(s):	Tenant
Description:	Tenant could be communicate with the landlord.
Triggering Event:	Tenant selects "Communicate."
Preconditions:	Tenant is logged in.
Assumptions:	Landlord will reply for the tenant message.
Main Path:	Tenant send messages to landlord. The platform will send notifications to tenant went the landlord reply.
Alternate Path:	If communication failed, the tenant will be notified to try again.
Postcondition:	Tenant and landlord be connected together.
Priority:	Medium

Case Name:	Cancel Rental
Case ID	TEN-05
Actor(s):	Tenant
Description:	Tenant could be cancel his rent request.
Triggering Event:	Tenant selects "Cancel Rental Request."
Preconditions:	Tenant is logged in.
Assumptions:	Tenant already has sent a rental request.
Main Path:	The landlord will be notified and state of house will be changed automatically. If it has not been 4 days from the date of the request, the warranty money will be returned to tenant account. If tenant exceeded the permitted period, the warranty money will be added to landlord balance.
Alternate Path:	If cancel failed, the tenant will be notified to try again.
Postcondition:	Tenant renting request be canceled and advertisement state be changed.
Priority:	Medium

Case Name:	Confirm Arrival
Case ID	TEN-06
Actor(s):	Tenant
Description:	Tenant could be able to confirm his arrival.
Triggering Event:	Tenant selects "Confirm Arrival."
Preconditions:	Tenant is logged in.
Assumptions:	Tenant arrived to rented house within the specified period and found that the house specification are identical to the advertisement. Landlord confirmed the user arrival request.
Main Path:	Tenant warranty money will returned to his balance. Balance amount will be updated.
Alternate Path:	If landlord did not confirm the arrival through 3 days, the warranty money will return back automatically to the tenant.
Postcondition:	Warranty money will return back to tenant balance.
Priority:	High

Case Name:	Request A House Review
Case ID	TEN-07
Actor(s):	Tenant
Description:	Tenant request customer service to review the house.
Triggering Event:	Tenant selects "House Review."
Preconditions:	Tenant is logged in.
Assumptions:	Tenant arrived to rented house within the specified period and found that the house specification does not match the advertisement.
Main Path:	Tenant warranty money will returned to his balance. Balance amount will be updated. Landlord will take a ban for 4 months.
Alternate Path:	If the complaint is false, the warranty money will be transformed to our system.
Postcondition:	Warranty money will return back to tenant balance and the landlord will be banned for 4 months.
Priority:	High

Case Name:	Set Rate
Case ID	TEN-08
Actor(s):	Tenant
Description:	Allows tenants to submit specific ratings and reviews about the house, the landlord, and a general overall rating of their rental experience.
Triggering Event:	Tenant completes a rental period, or the landlord finalizes the rental agreement.
Preconditions:	Tenant has completed a stay at the property. Both parties are logged in to their accounts.
Assumptions:	The review system includes fields to rate the house, the landlord, and a general experience score. Users are encouraged to provide honest, respectful feedback. Protections are in place to detect inappropriate or biased reviews.
Main Path:	Tenant accesses the "Rate and Review" section after completing their rental stay. Tenant provides a separate rating for the house, the landlord, and a general rating. • House Rating: Tenant rates the property's condition, amenities, and match with the listing description. • Landlord Rating: Tenant rates the landlord's responsiveness, professionalism, and communication. • General Rating: Tenant provides an overall rating for the entire experience. Tenant submits additional comments or feedback. Platform publishes the ratings and review, displayed on the profiles of the tenant, landlord, and property listing.
Alternate Path:	If a review is flagged for inappropriate content, it is temporarily hidden and sent for review by customer service.
Postcondition:	Ratings and reviews are saved, helping future users make informed decisions based on house, landlord, and general experience ratings.
Priority:	Medium

Case Name:	Reviews Complains
Case ID	CS-01
Actor(s):	Customer Service
Description:	Customer service receives a review request from tenant.
Triggering Event:	Tenant selects "Review complain".
Preconditions:	Customer service agent has been added to the system.
Assumptions:	Customer service is online to receive the notification.
Main Path:	If there is really a difference between the house and advertisement, the landlord will be blocked for 4 months any warranty money will be transformed back to tenant balance without landlord confirmation. If it is a false complaint, the warranty money will be transformed to the system.
Alternate Path:	If a customer service agent is offline when the complaint occurs, they will be notified as soon as they are online again.
Postcondition:	Landlord be blocked or warranty money be transformed to the system.
Priority:	High

Case Name:	Review/Enter House Information
Case ID	CS-02
Actor(s):	Customer Service
Description:	Customer service inspects a submitted property and defines specifications.
Triggering Event:	Landlord submits a rental request.
Preconditions:	Landlord's property request is valid and verified.
Assumptions:	Customer service team has adequate resources for inspections. Inspection results are objective and accurate, preventing false listings.
Main Path:	Customer service schedules and performs inspection. They define property specifications (rooms, amenities, photos). Advertisement is published on the platform.
Alternate Path:	If additional proof is needed, customer service requests it from the landlord.
Postcondition:	Property is listed with verified details.
Priority:	High

Case Name:	Forgot Password
Case ID	GEN-01
Actor(s):	(Tenant , Landlord, Customer Service Agent)
Description:	Allows users (tenants, landlords, Customer Service Agents) to reset their password if they cannot remember it.
Triggering Event:	User selects the "Forgot Password" option on the login screen.
Preconditions:	User has an account on the platform with a registered email or phone number. User can access their email or phone for verification.
Assumptions:	The email or phone number is correctly linked to the user's account. The user's registered email service or phone carrier is operational. The system has implemented proper security to handle password resets securely.
Main Path:	User selects "Forgot Password" on the login page. Platform prompts user to enter their registered email or phone number. Platform sends a password reset link (or verification code) to the provided email or phone. User receives the reset link or code and follows instructions (clicking link or entering code). Platform verifies the link or code and prompts user to enter a new password. User enters a new password and confirms it. Platform updates the user's password and confirms the reset.
Alternate Path:	If the entered email or phone number does not match any account, platform displays an error message. If the reset link or code expires, platform notifies the user to request a new one.
Postcondition:	User's password is successfully updated, and they can log in with the new password.
Priority:	High

Case Name:	Edit profile
Case ID	GEN-02
Actor(s):	(Tenant , Landlord, Customer Service Agent)
Description:	Allows users (tenants, landlords, Customer Service Agents) to edit their profile information.
Triggering Event:	User selects the "Edit profile" option.
Preconditions:	User has an account on the platform with a registered email or phone number.
Assumptions:	User profile information is not updated.
Main Path:	User selects the field that he want to change. User enters the new information. Platform saves the changes to the profile.
Alternate Path:	If the user entered invalid information for the selected field, the system will notify him.
Postcondition:	User profile is up to date.
Priority:	High

Case Name:	User Login
Case ID	GEN-03
Actor(s):	User (Tenant, Landlord, or Customer Service Agent)
Description:	Allows any user (tenant, landlord, or customer service agent) to securely log in to their account on the platform.
Triggering Event:	User accesses the login page and enters credentials.
Preconditions:	User has an existing account with registered email/username and password. The user's device is compatible with the platform.
Assumptions:	Login process includes security measures such as CAPTCHA or multi-factor authentication for authorized access. System enforces lockout policies after multiple failed login attempts.
Main Path:	User enters their email/username and password on the login page. Platform validates credentials. If credentials are correct, the user is granted access to their account.
Alternate Path:	If credentials are incorrect, the user receives an error message and an option to retry.
Postcondition:	User successfully logs in and accesses their account dashboard.
Priority:	High

Case Name:	Deposit
Case ID	GEN-04
Actor(s):	User (Tenant or Landlord)
Description:	Allows any user (tenant or landlord) to deposit money into his balance.
Triggering Event:	User selects "Deposit".
Preconditions:	User logged in.
Assumptions:	Payment Gateway is secure.
Main Path:	User chooses the deposit method (Credit or electronic wallet). User enter the required information. The amount of money is deposited to the use balance.
Alternate Path:	If the Credit or electronic wallet required information is wrong, the system will notify user.
Postcondition:	The deposit process is completed successfully.
Priority:	High

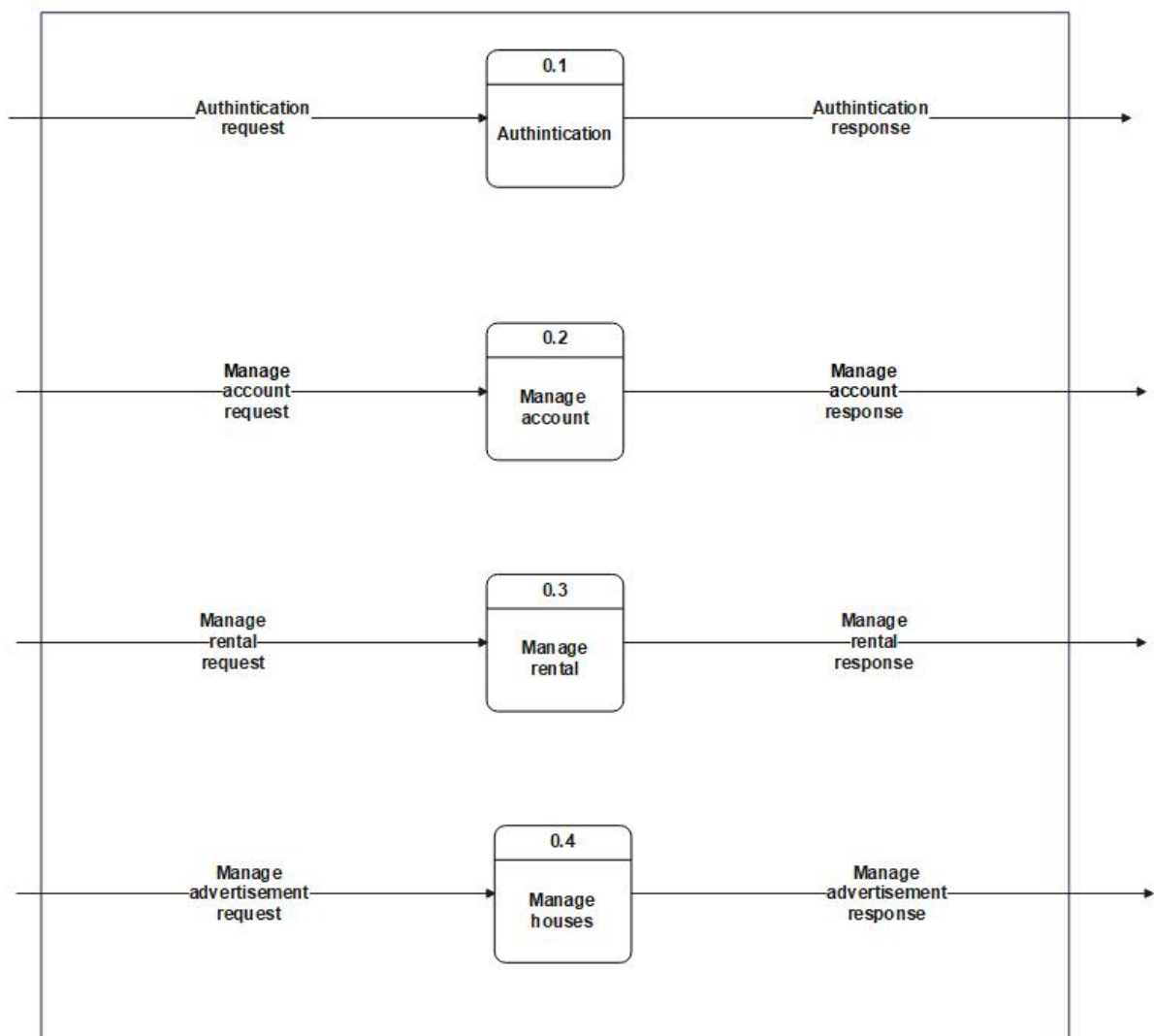
Case Name:	Withdraw
Case ID	GEN-05
Actor(s):	User (Tenant or Landlord)
Description:	Allows any user (tenant or landlord) to withdraw money from his balance.
Triggering Event:	User selects "Withdraw".
Preconditions:	User logged in.
Assumptions:	Payment Gateway is secure.
Main Path:	User chooses the withdraw method (Credit or electronic wallet). User enter the required information. The amount of money is withdrawn from the use balance.
Alternate Path:	If the Credit or electronic wallet required information is wrong, the system will notify user. If the user balance is less than the amount to be withdrawn, the system will notify the user
Postcondition:	The withdrawal process is completed successfully.
Priority:	High

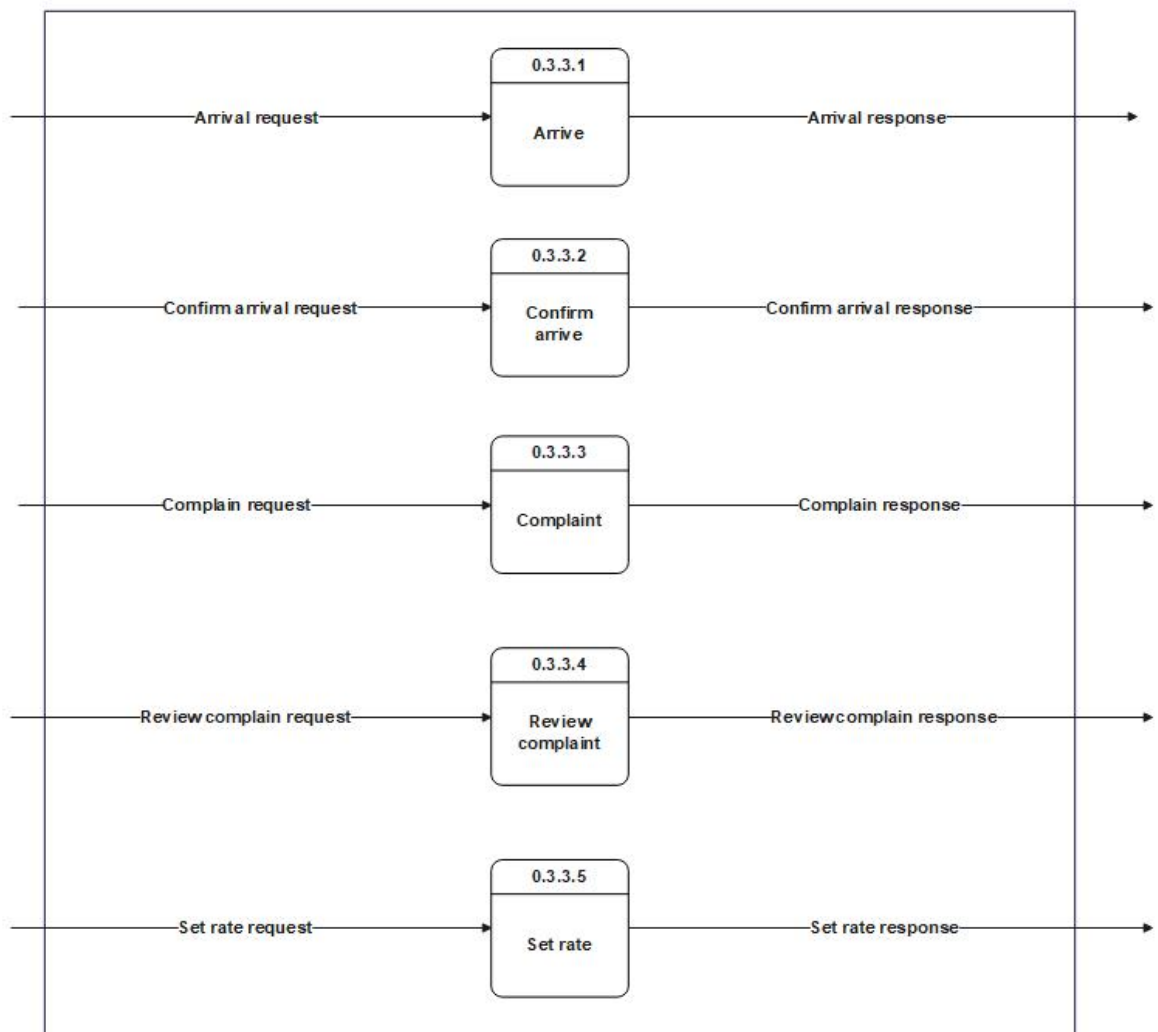
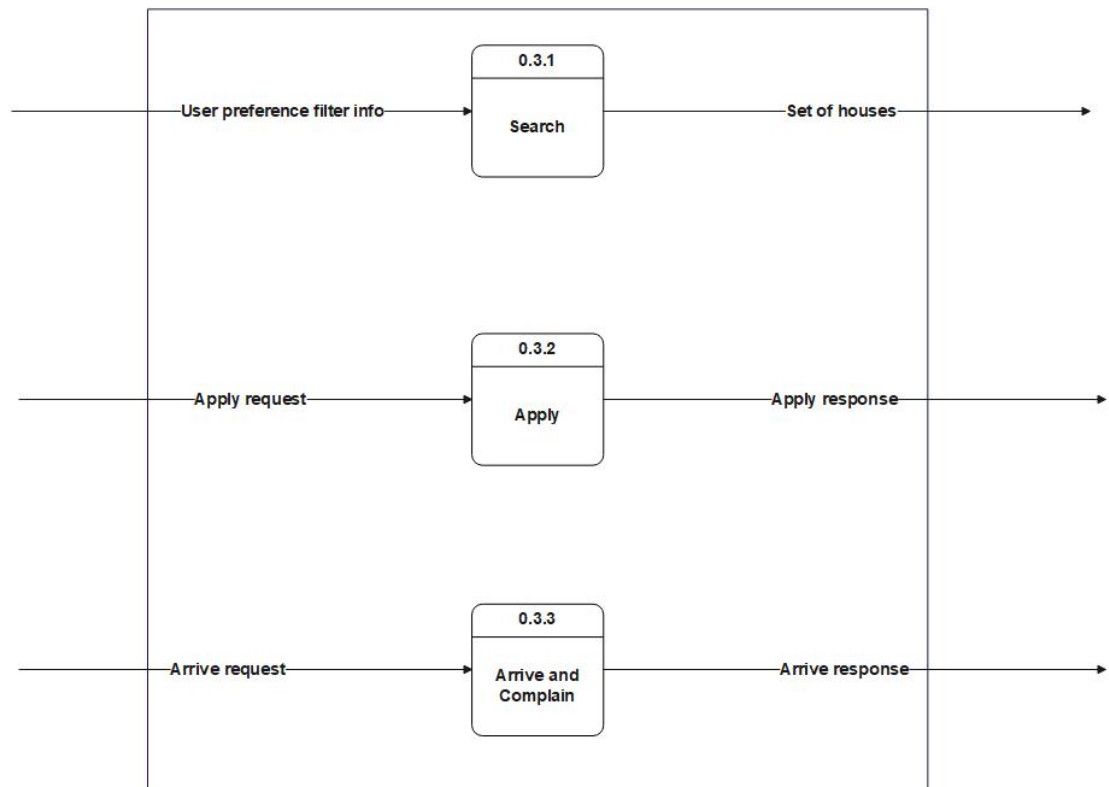
4.3 System Architecture

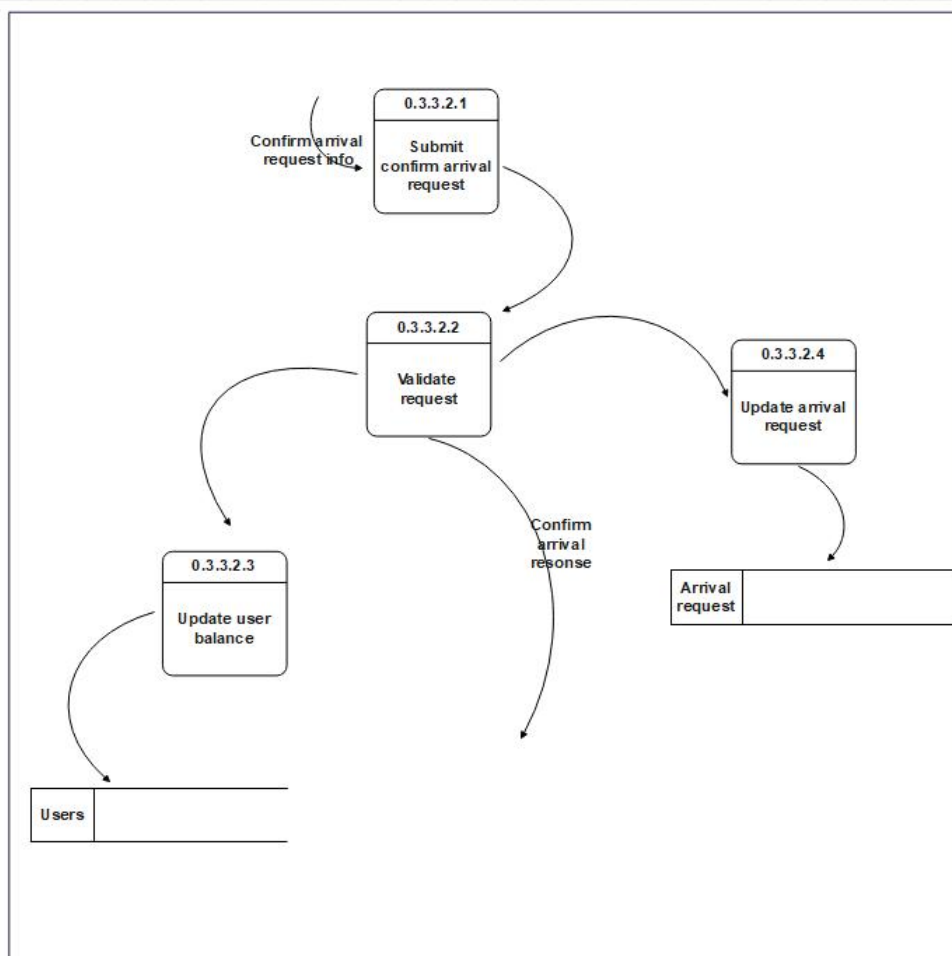
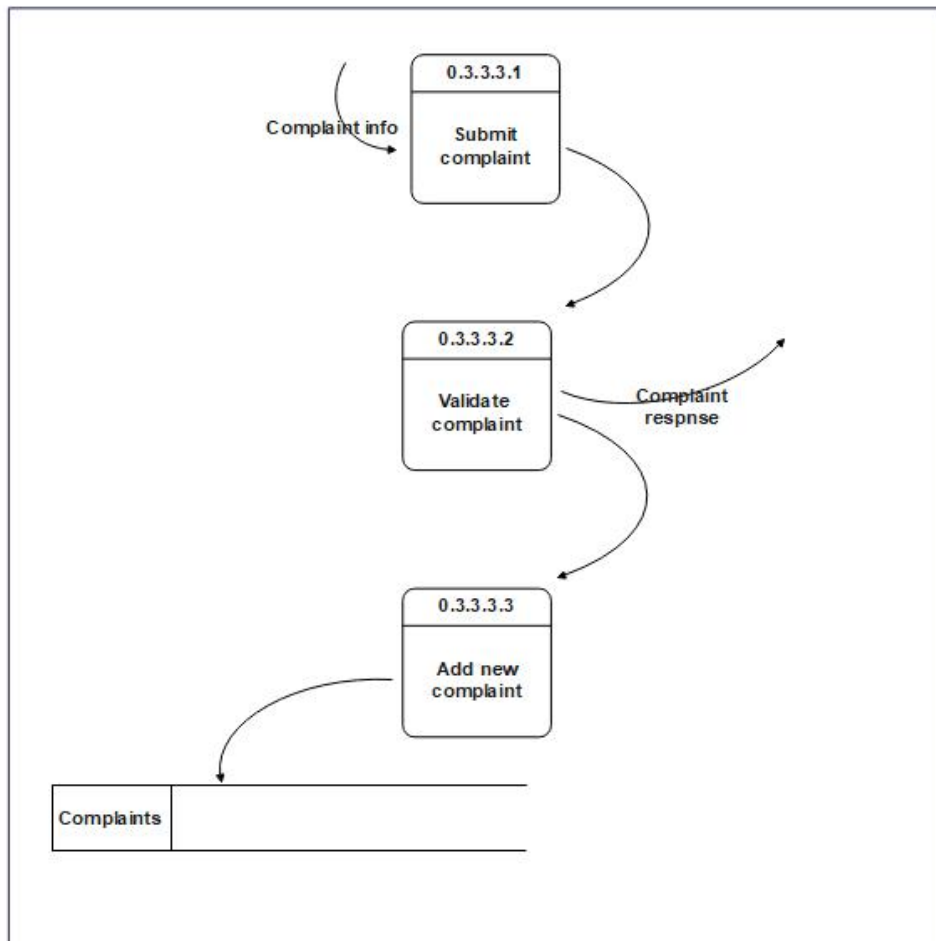
4.4 Analysis Class

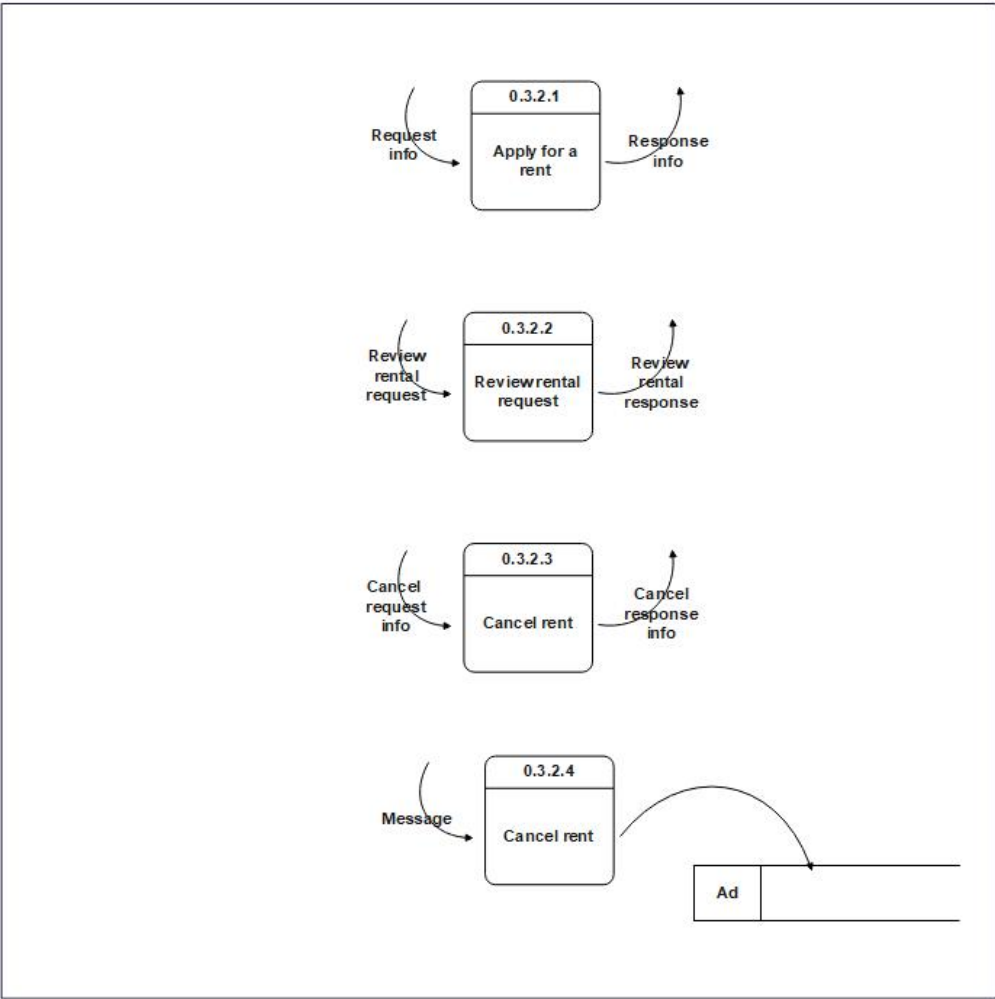
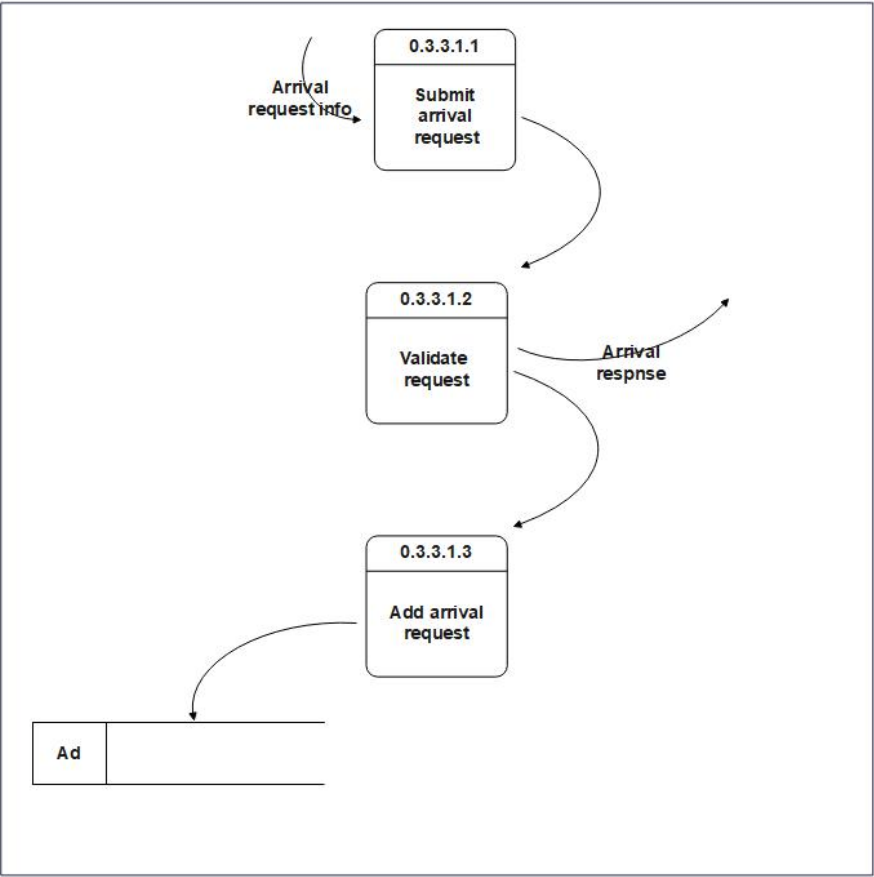
4.4.1 State Diagram

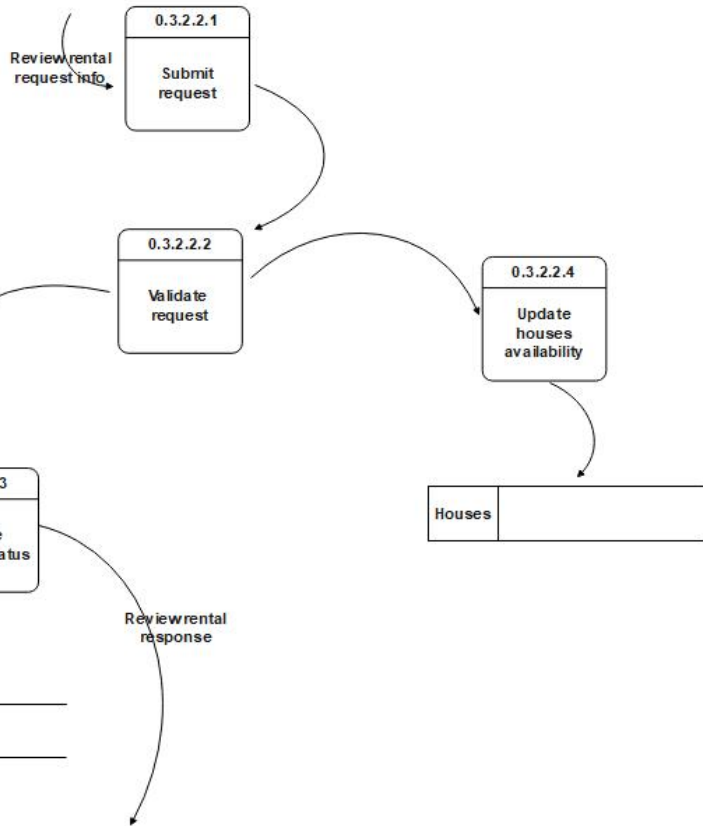
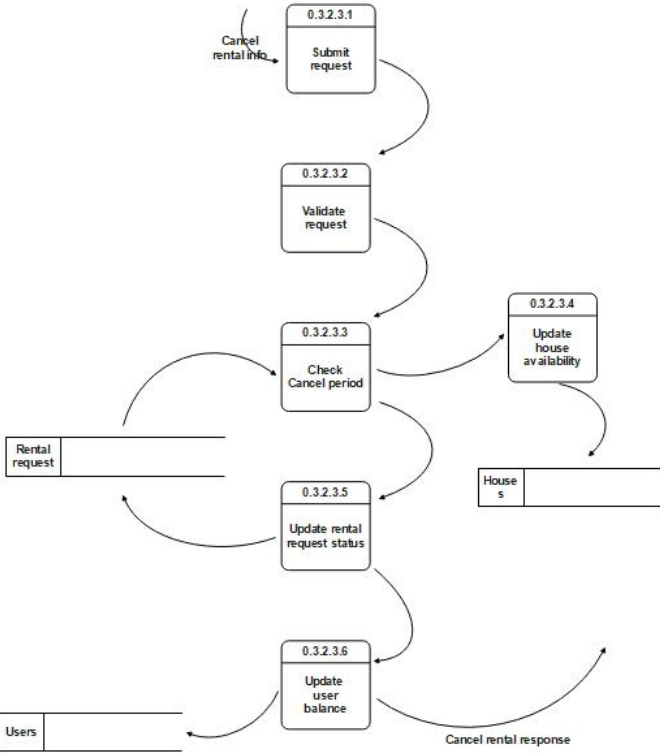
4.4.2 Data flow diagram (Level Zero diagram)

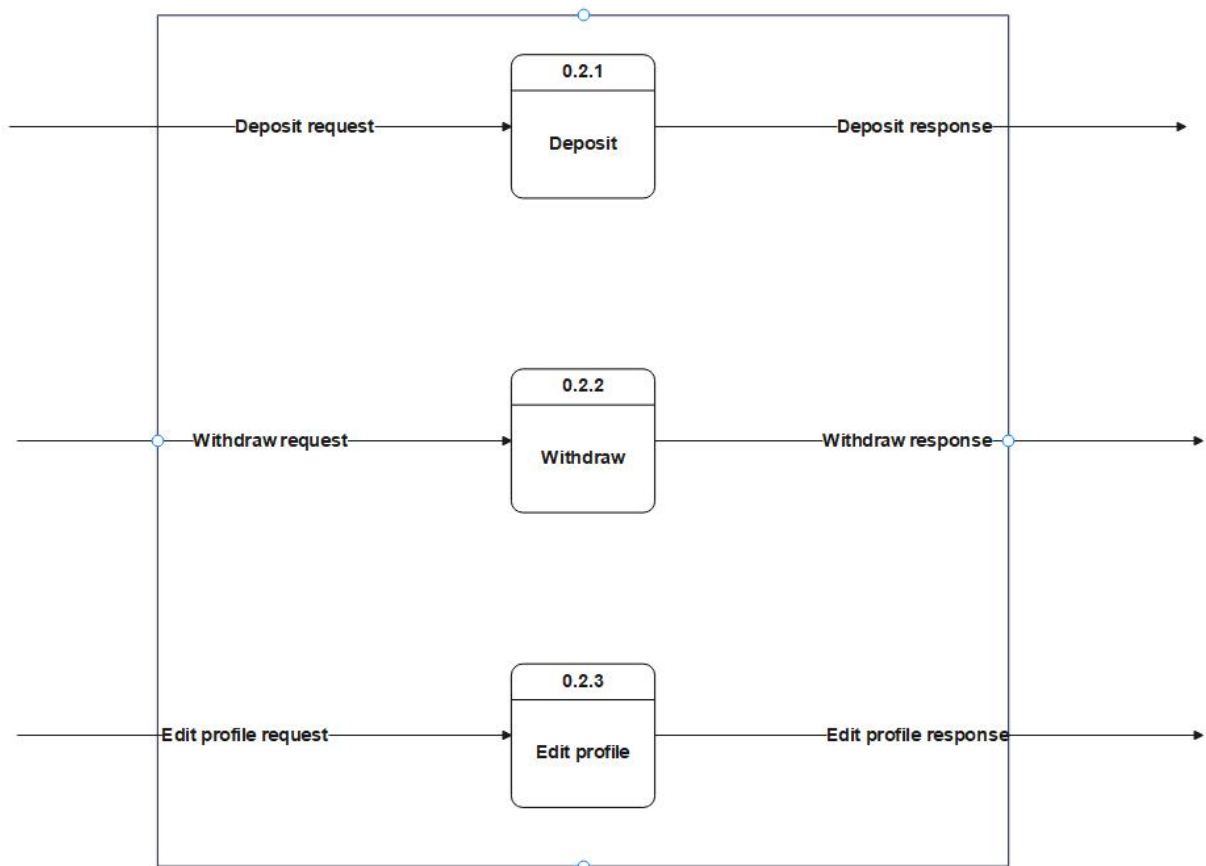
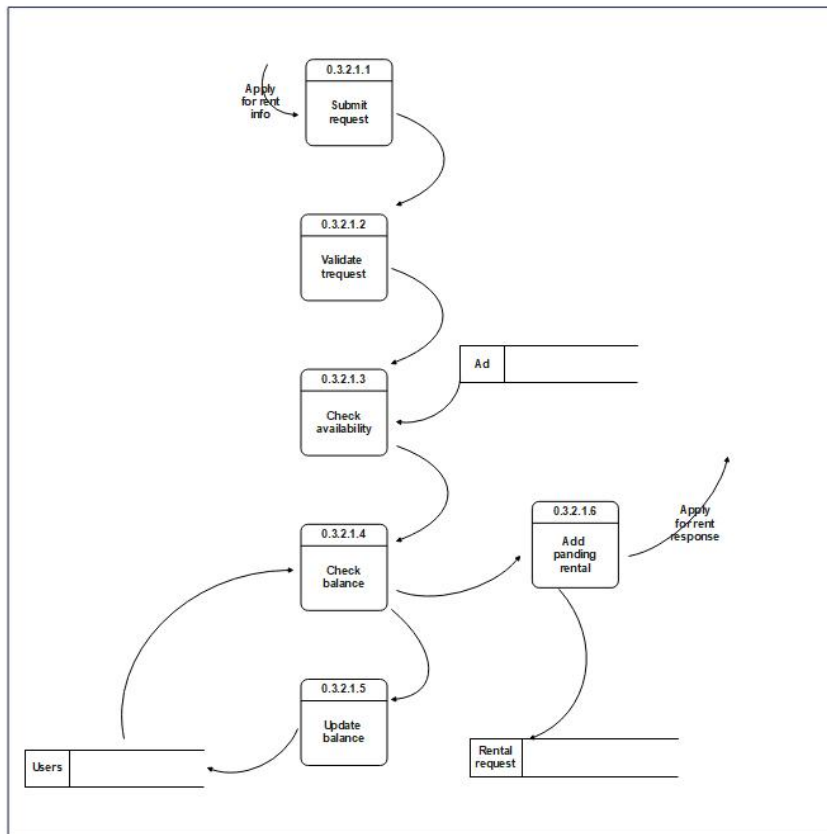


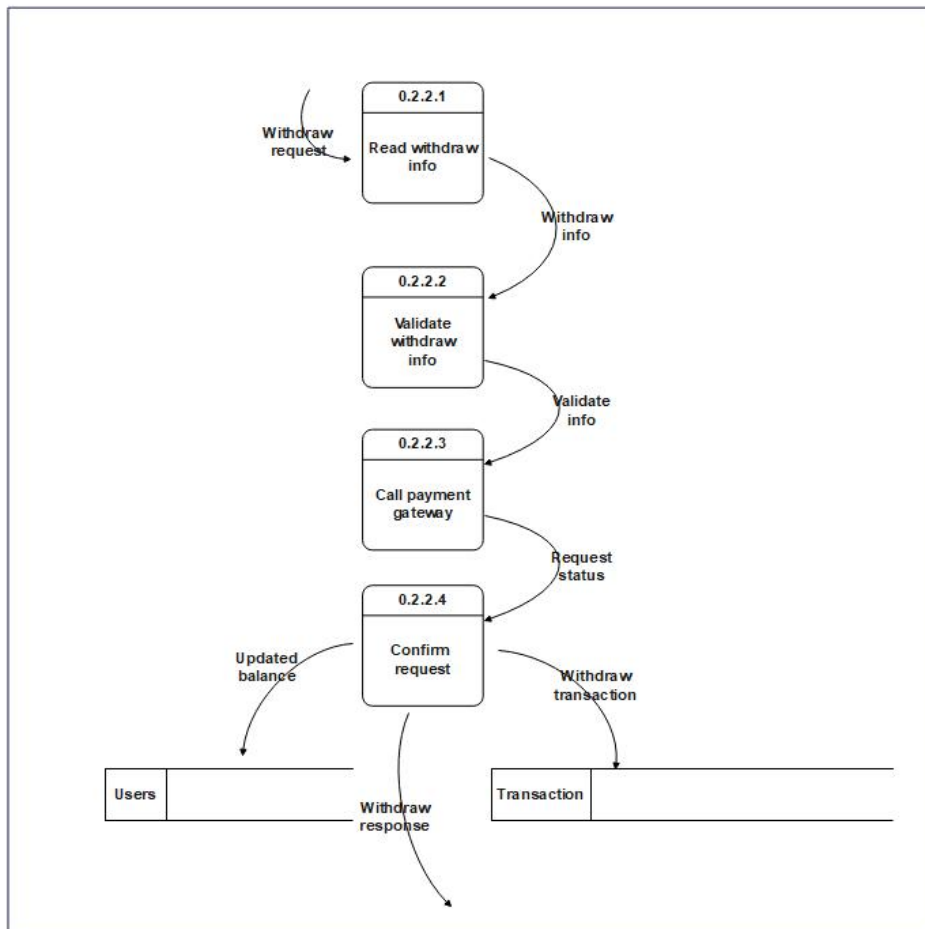
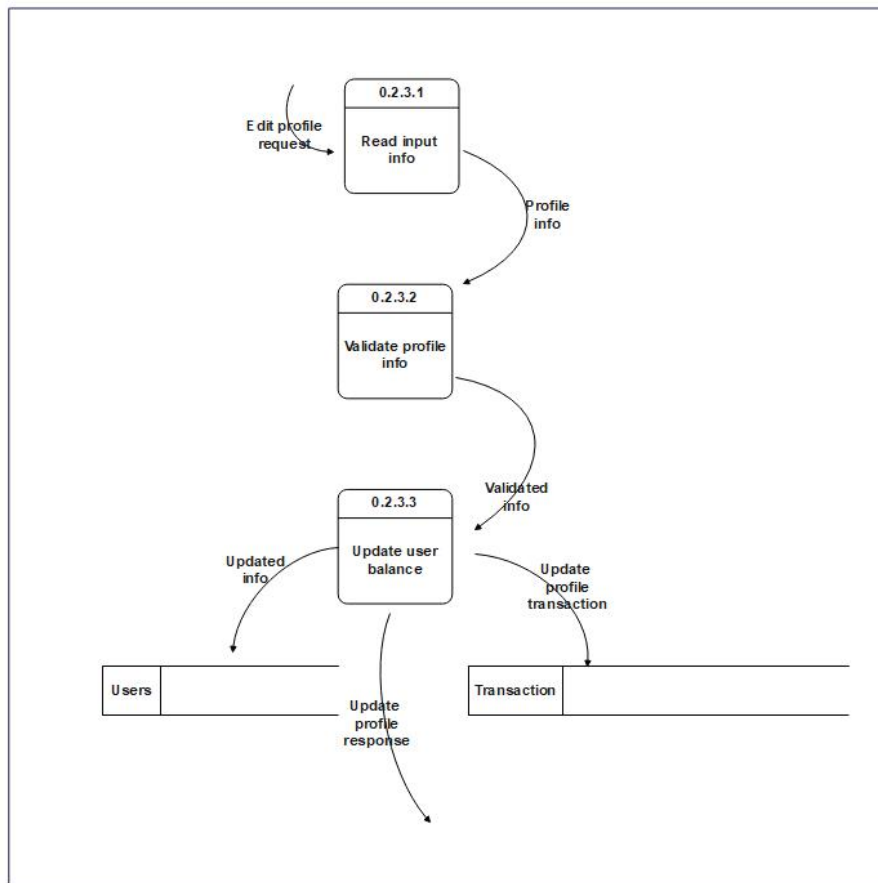


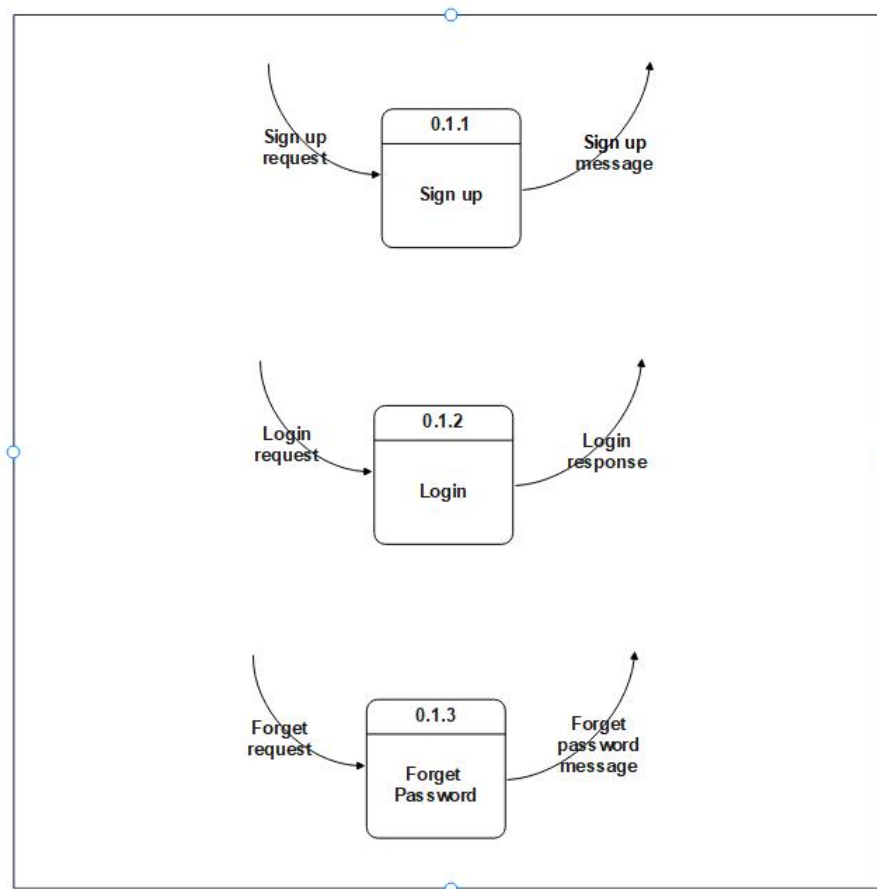
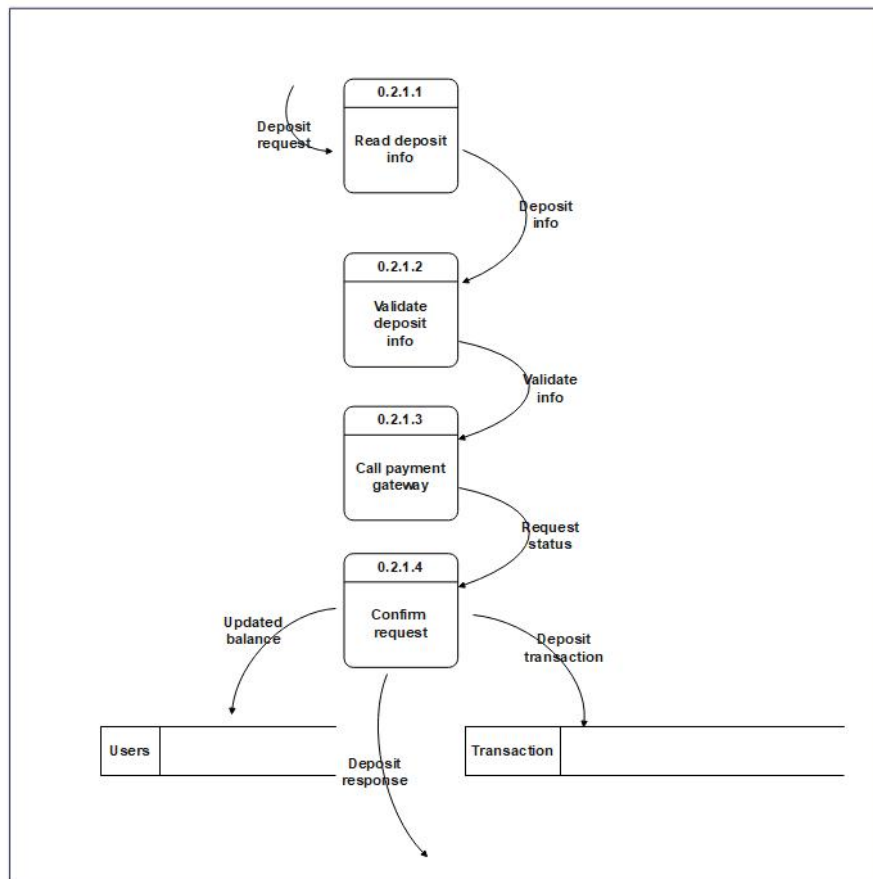


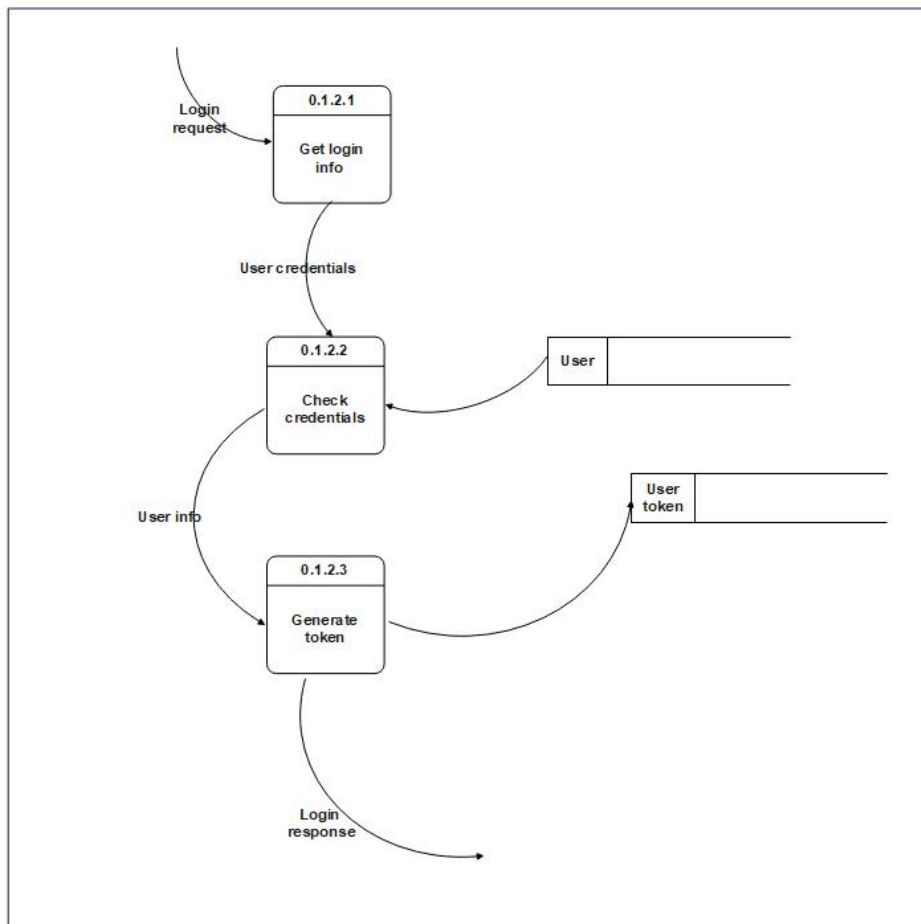
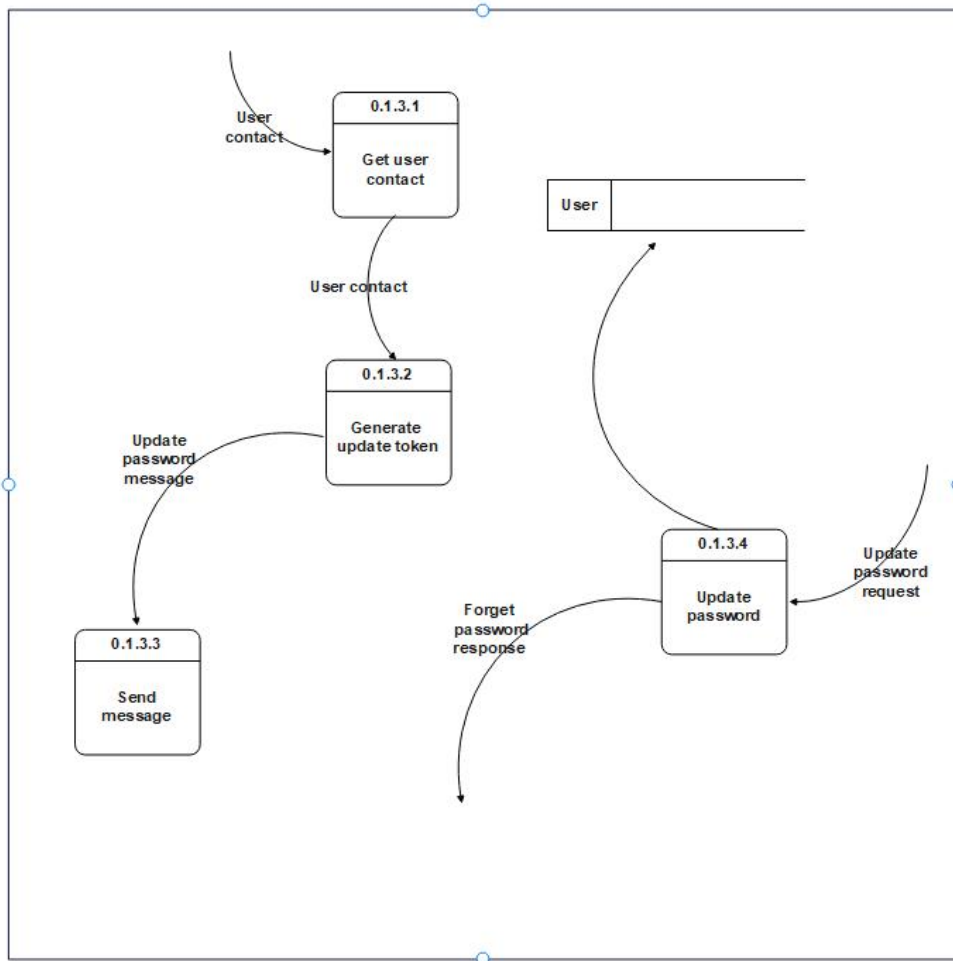


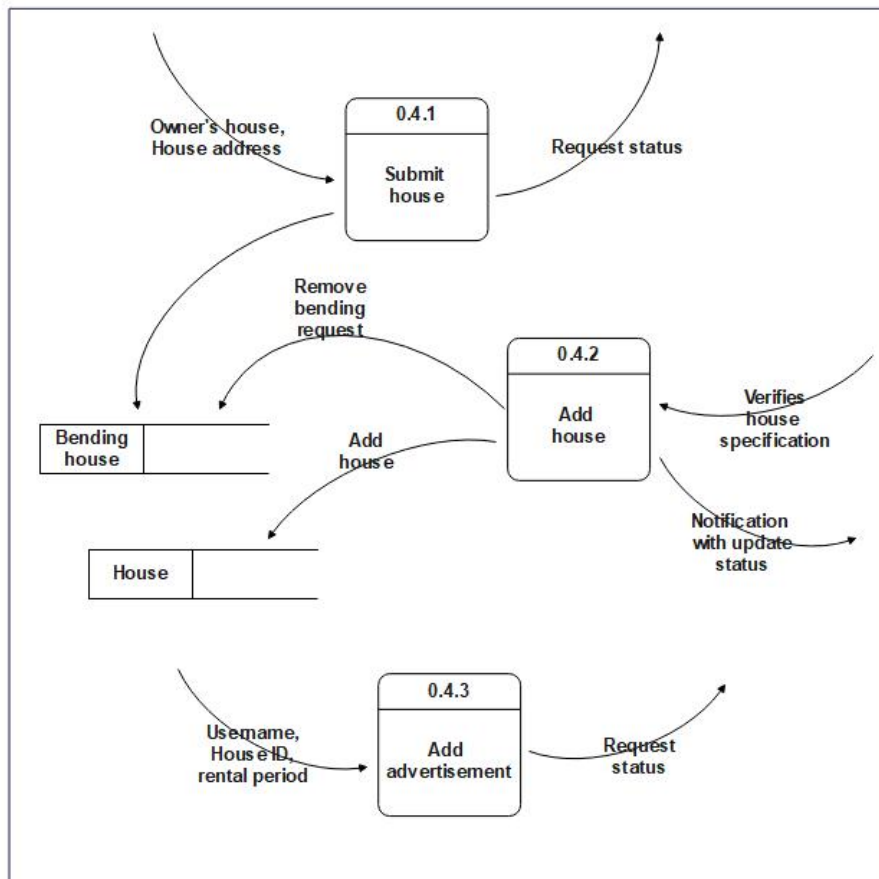
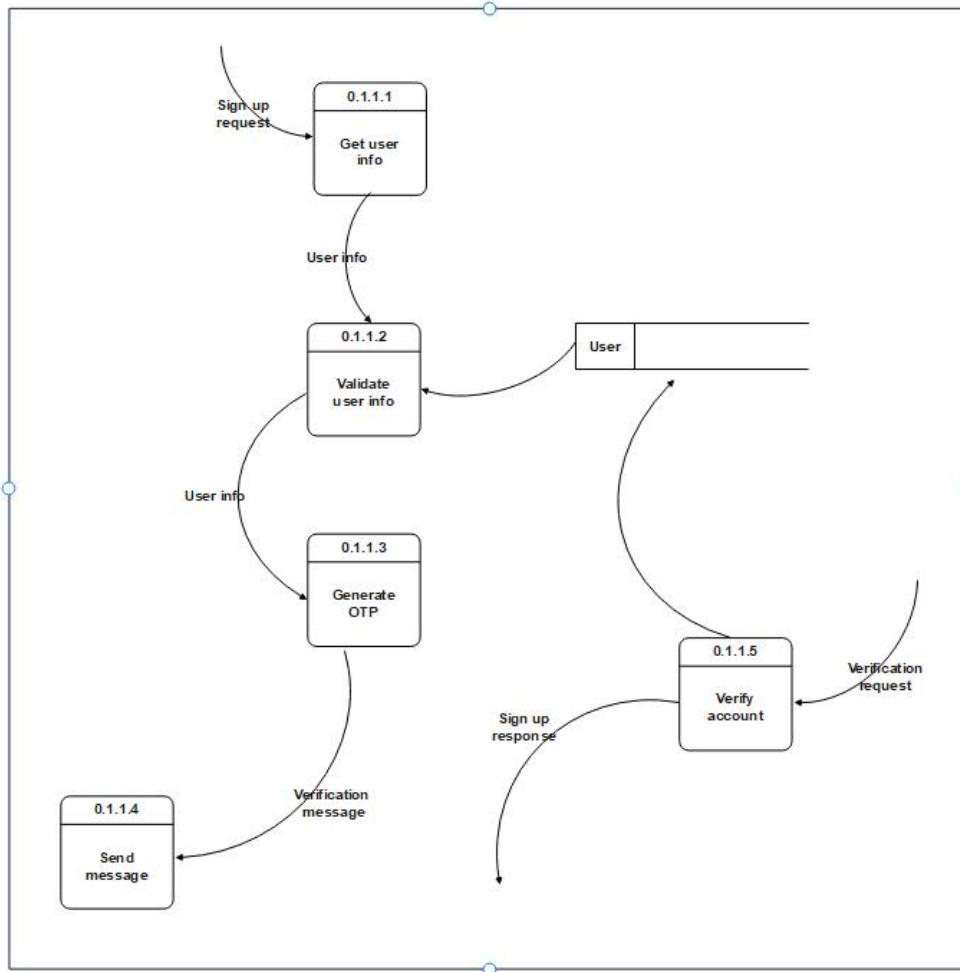




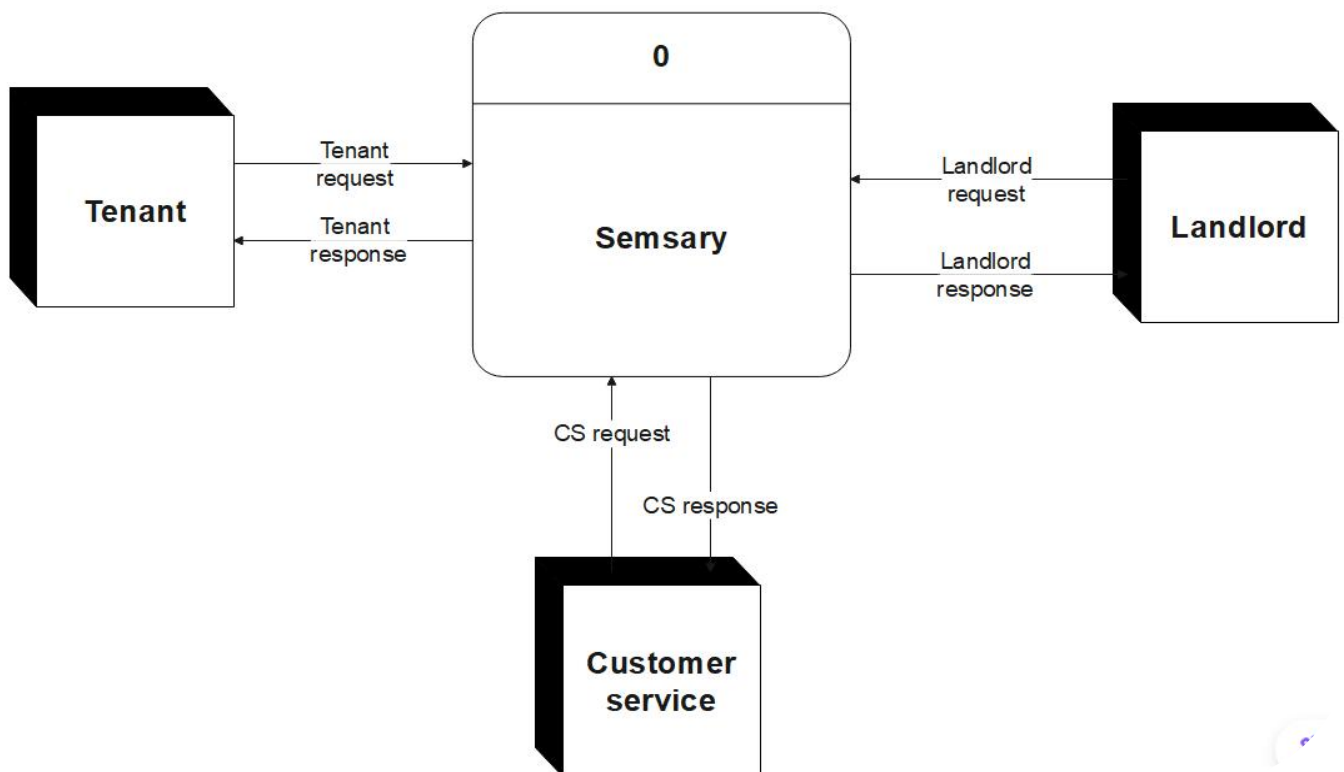






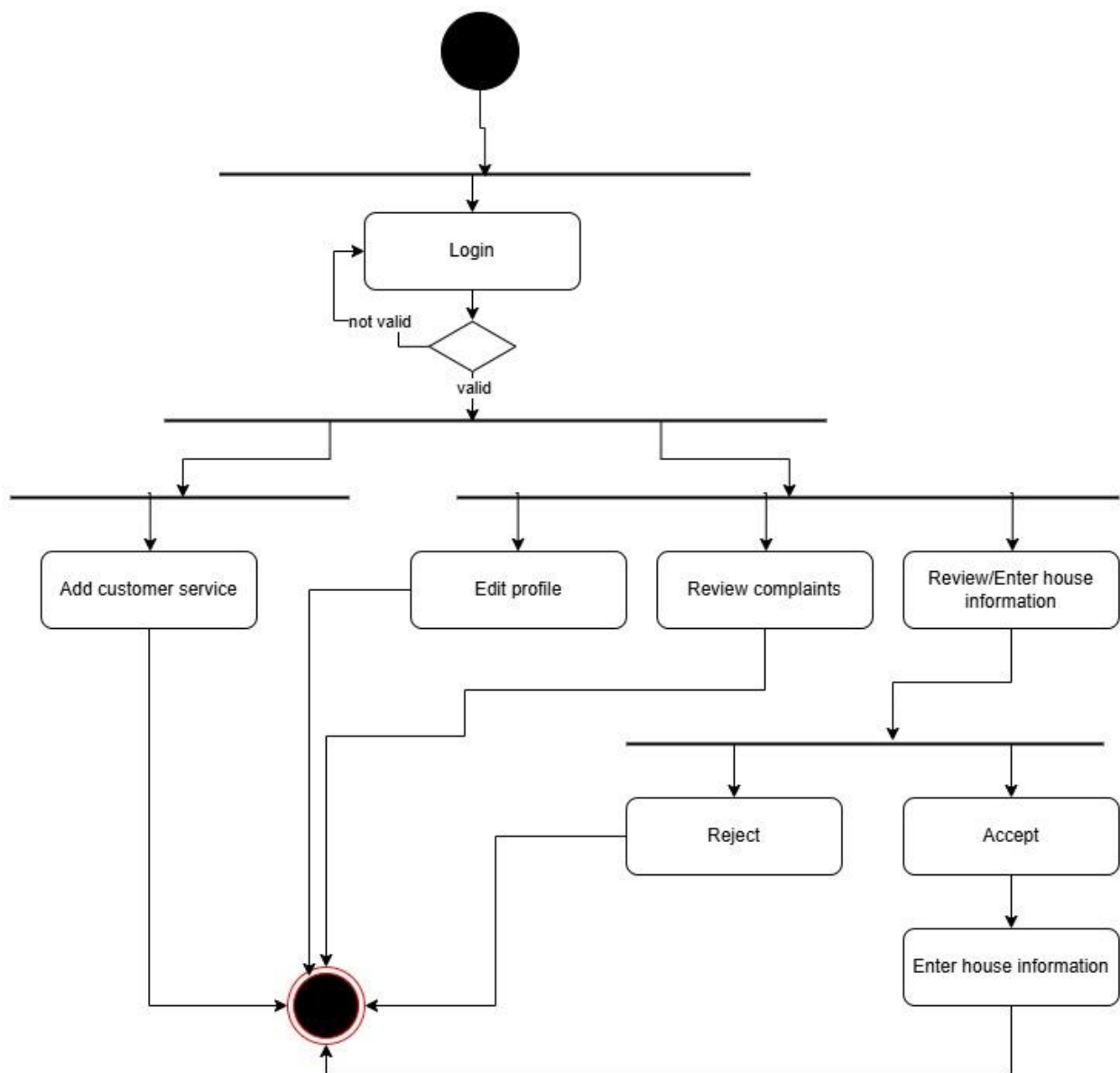


4.4.3 Context Diagram

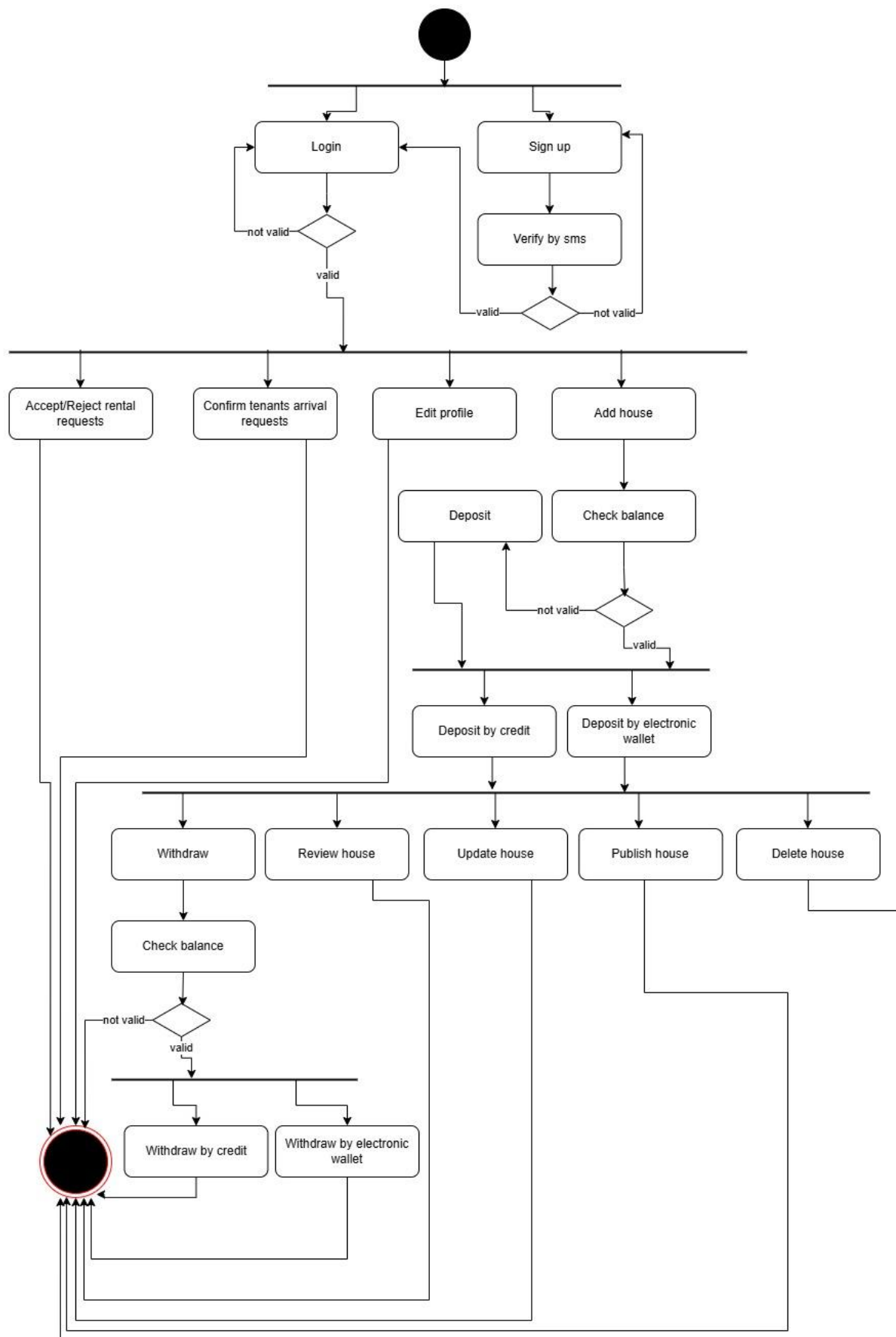


4.5 Activity diagrams

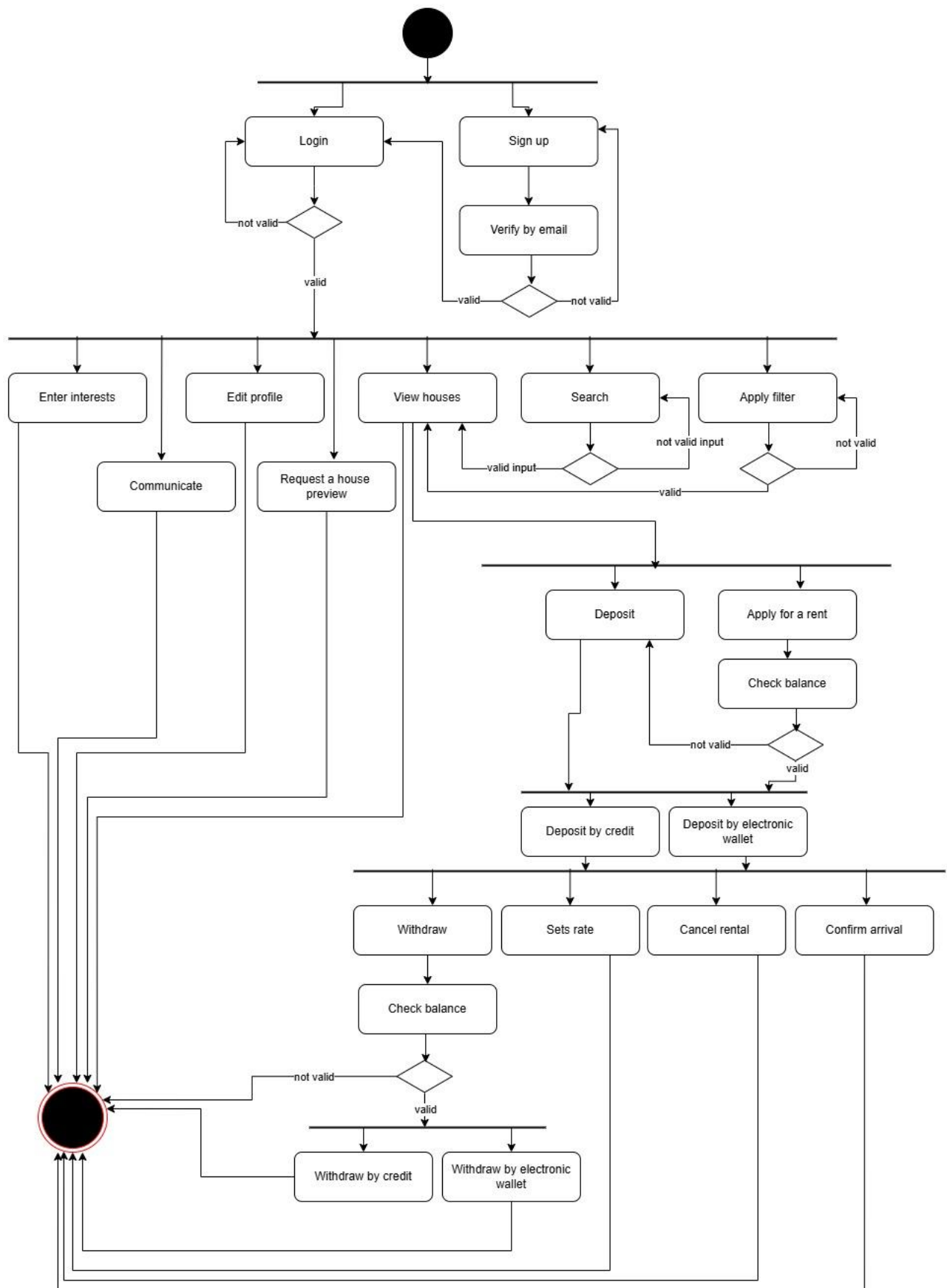
4.5.1 Admin



4.5.2 Landlord



4.5.3 Tenant



4.6 interaction class diagram

4.6.1 System diagram

4.6.2 Sequence Diagram

4.7 Design Class

4.7.1 Class Diagram

4.8 Database Schema

4.9 ER Diagram

4.10 Design Mockup



Chapter 5

Proposed System & Methodology

5.1 Conclusions

5.2 Future work

5.3 References